

What *Learning* Means

Contents

Notation (list of symbols)	3
1 What this chapter covers	5
2 The loss function	6
2.1 The setup: model, prediction, target	6
2.2 The two standard losses	6
2.3 The loss defines what “good” means	8
2.4 The loss landscape	8
2.5 Symbols	10
3 Gradient descent	11
3.1 The update rule	11
3.2 A descent computed by hand	11
3.3 Where the walk ends	13
3.4 Why this simple method	13
3.5 Symbols	14
4 The learning rate	15
4.1 What the learning rate controls	15
4.2 One landscape, six learning rates	15
4.3 Reading the loss curve	17
4.4 Symbols	17
5 Stochastic gradient descent	19
5.1 The cost problem	19
5.2 The fix: average over a random sample	20
5.3 One step in full: what is read, what is ignored	20
5.4 The noise, shown by hand	22
5.5 Epochs and practice	24
5.6 Symbols	25
6 Overfitting and underfitting	27
6.1 The goal is new data	27
6.2 One dataset, three models	27
6.3 The verdict of new data	28
6.4 Capacity, and telling the failures apart	29
6.5 Symbols	30
7 The bias–variance trade-off	31
7.1 Same family, different draw	31
7.2 Bias and variance	32
7.3 The decomposition	32
7.4 The trade-off	34
7.5 Symbols	34
8 Train, validation, and test splits	35

8.1	Creating new data by holding some back	35
8.2	Why two parts are not enough	35
8.3	The leak, computed	35
8.4	Splits in practice	37
8.5	Symbols	38
9	Generalization	39
9.1	The measure	39
9.2	The assumption that makes it possible	39
9.3	When the assumption breaks: distribution shift	39
9.4	The levers	40
9.5	Symbols	40
10	Regularization	42
10.1	Complexity is visible in the parameters	42
10.2	L2: penalize the squares	42
10.3	L1: penalize the absolute values	43
10.4	The dial, demonstrated	43
10.5	Symbols	44
11	The map of machine learning	46
11.1	Supervised learning	46
11.2	Unsupervised learning	46
11.3	Reinforcement learning	47
11.4	What transfers	47
11.5	Symbols	47
12	Evaluation metrics	49
12.1	One thousand emails	49
12.2	The confusion matrix	49
12.3	Precision, recall, F1	50
12.4	The threshold, and the trade-off	50
12.5	Choosing the number	51
12.6	Symbols	51

Notation (list of symbols)

θ	the model's parameters: the values in its formula left free to set, collected in one vector.
$L(\theta)$	the loss: one number measuring how wrong the model currently is, as a function of the parameters.
x	an input example, as a vector of features.
y	the target (label): the correct answer for input x .
$\hat{y}$	the model's prediction for input x : $\hat{y} = f_{\theta}(x)$.
∇L	the gradient of the loss: the vector of all partial derivatives $\partial L / \partial \theta_i$; points steepest uphill, its negative steepest downhill.
η	the learning rate: the step size of gradient descent.

Convention — indexing. Unless a section says otherwise, indices are 1-*based*: a vector $x \in \mathbb{R}^n$ has entries $x_1, \dots, x_n$, and matrix rows and columns count from 1. Where 0-based indexing is the natural choice (token positions, anything mirroring code), the section says so explicitly.

1 What this chapter covers

The core loop of all of machine learning: a model is just *a function with tunable parameters*; a *loss function* reduces “how wrong is it?” to a single number; and *learning* means adjusting the parameters to shrink that number. Everything else in the field is elaboration on this loop.

The concepts (★ = load-bearing):

- **Loss function ★** — the objective being minimized (mean squared error — MSE — for regression, cross-entropy for classification); the loss *defines* what “good” means. (§2)
- **Gradient descent ★** — compute the gradient, step the opposite direction, repeat; how every neural network on Earth is trained. (§3)
- **Learning rate** — the step size: too large and training diverges, too small and it barely moves. The most important hyperparameter. (§4)
- **Stochastic / mini-batch gradient descent** — estimate the gradient from a small random batch; noisier but vastly cheaper, and the noise even helps generalization. (§5)
- **Overfitting vs. underfitting ★** — memorizing the training data vs. being too simple to capture the pattern. (§6)
- **Bias–variance trade-off** — the classic frame for that balance: underfitting is too much bias, overfitting is too much variance. (§7)
- **Train / validation / test splits** — train on one, tune on the second, report on a third the model never influenced. (§8)
- **Generalization** — performance on unseen data: the actual goal. (§9)
- **Regularization (L1/L2)** — penalize large weights; favors simpler, more general functions. (§10)
- **The ML taxonomy** — supervised / unsupervised / reinforcement learning; classification vs. regression. (§11)
- **Evaluation metrics** — accuracy, precision, recall, F1, ROC; 99% accuracy is meaningless if 99% of emails are not spam. (§12)

2 The loss function

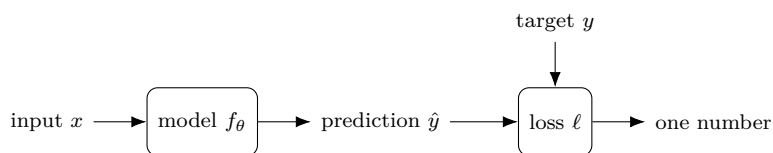
A model is a function with adjustable *parameters*: fed an input, it returns a prediction, and *which* prediction it returns depends on the current parameter values. (Parameters are not the function’s argument — the input x is. Parameters are the values inside the formula: fixed across inputs, changed only when training adjusts them.) Before anything can be learned, one question must become measurable: *how wrong is the model right now?* The loss function is the formula that answers it. Everything else in this chapter — gradient descent, overfitting, regularization — operates on the number it produces.

2.1 The setup: model, prediction, target

Three objects, then the definition:

- f_θ is the model: a function whose behavior is set by its parameters θ (all of them collected in one vector).
- $\hat{y} = f_\theta(x)$ is the *prediction*: the model’s answer for the input x . The hat marks “the model’s estimate”.
- y is the *target* (also *label*): the correct answer for x , known during training.

A *loss function* is a formula that measures how far the prediction $\hat{y}$ is from the target y , as a single nonnegative number. Zero means a perfect prediction; larger means worse. Write $\ell(\hat{y}, y)$ for one example’s loss:



Averaged over the N training examples, the per-example losses become the training loss — a function of the parameters, since changing θ changes every prediction:

$$L(\theta) = \frac{1}{N} \sum_{n=1}^N \ell(\hat{y}_n, y_n), \quad \hat{y}_n = f_\theta(x_n)$$

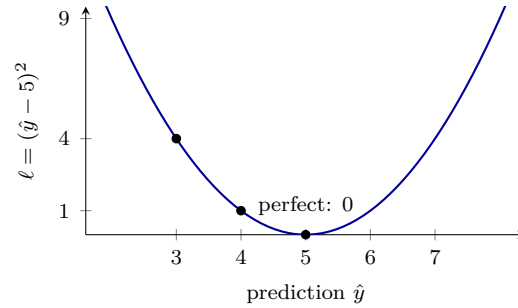
— the prediction and the target of the n -th example. This $L(\theta)$ is the single number training works to reduce. One requirement is already visible: ℓ must be *differentiable*, because training will need the gradient ∇L (the vector of all partial derivatives of L) — whose *negative* is the direction in which the wrongness decreases.

2.2 The two standard losses

Predicting a number (regression): mean squared error. The per-example loss is the squared gap,

$$\ell(\hat{y}, y) = (\hat{y} - y)^2.$$

Worked, with target $y = 5$: predicting $\hat{y} = 4$ gives loss 1; predicting $\hat{y} = 3$ gives loss 4. Being twice as far off gives four times the loss — the square punishes large errors far more than small ones:



Averaged over examples this is the *mean squared error* (MSE). It is not an arbitrary choice: the maximum-likelihood section of *Essential Math for AI* derived it — assume Gaussian noise on the observations and maximize likelihood, and minimizing MSE is what falls out.

Predicting a class (classification): cross-entropy. Classification is the task of choosing one option from a fixed list — say cat, dog, or bird. Each option is called a *class*, and K is the number of classes ($K = 3$ here).

The loss $\ell(\hat{y}, y)$ still takes the same two inputs as in the setup above — the prediction and the target — but in classification both look different from regression:

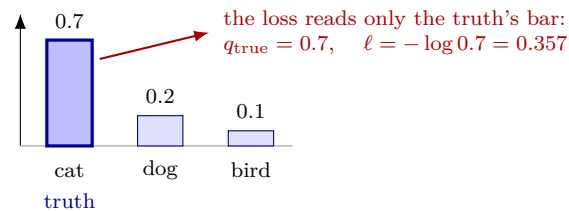
- The target y is the correct class — here, “cat”. It is a choice from the list, not a number.
- The prediction $\hat{y}$ is a list of K probabilities, one per class,

$$q = (q_{\text{cat}}, q_{\text{dog}}, q_{\text{bird}}) = (0.7, 0.2, 0.1),$$

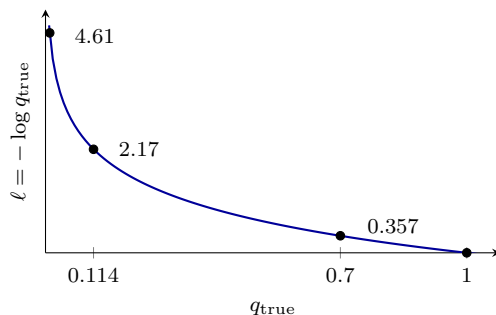
with entries nonnegative and summing to 1 ($0.7 + 0.2 + 0.1 = 1$). A list with these two properties is a *probability distribution*, and the output layer that produces it is the *softmax* — so a prediction of this kind is called a *softmax distribution*, the term used from here on.

The loss connects the two in the simplest possible way: ignore every entry of q except the one belonging to the correct class. Read that number out — call it q_{true} ; here $q_{\text{true}} = q_{\text{cat}} = 0.7$ — and take its *surprise*, $-\log$ of it (the term from the entropy section of *Essential Math for AI*):

$$\ell = -\log q_{\text{true}}.$$



The growth of this loss is the classification counterpart of the parabola: probability 1 on the correct class gives loss 0, and the loss grows without bound as the probability given to the correct class approaches 0:



The three marked points, as situations:

Situation	q_{true}	Loss $-\log q_{\text{true}}$
confident and right	0.7	0.357
unsure	0.114	2.17
confident and wrong	0.01	4.61

This is the *cross-entropy loss*, and it too is derived, not chosen: it is the negative log-likelihood of the observed labels (*Essential Math for AI* builds the identity in full). Confidence in the wrong answer is what it punishes hardest.

Task	Output	Loss	Derived from
regression	a number	MSE $(\hat{y} - y)^2$	Gaussian noise + MLE
classification	a distribution	cross-entropy $-\log q_{\text{true}}$	NLL of the labels

2.3 The loss defines what “good” means

The loss is not a neutral measurement — it is the *definition of the goal*: the same network trained on the same data becomes a different tool under a different loss.

The choice of task is not the only thing the loss decides. The *curve* of the per-example loss ℓ — how fast the loss grows as the prediction $\hat{y}$ moves away from the target y — also shapes the model’s behavior. A prediction can miss in two directions: $\hat{y} < y$ is called an *under-prediction*, $\hat{y} > y$ an *over-prediction*. The parabola $\ell = (\hat{y} - y)^2$ of the previous subsection treats the two directions equally: with target $y = 5$, the under-prediction $\hat{y} = 3$ and the over-prediction $\hat{y} = 7$ both give loss 4.

Nothing forces this symmetry; it is simply a property of the squared error. A *different* loss — not the MSE just plotted — can be designed with a steeper curve on the under-prediction side: it would give $\hat{y} = 3$ a larger loss than $\hat{y} = 7$, and a model trained under it would learn to prefer predicting high. Such *asymmetric* losses are chosen when the two mistakes differ in reality: in stock planning, running out is worse than having extra.

2.4 The loss landscape

This subsection builds the picture that the rest of the chapter stands on: the *loss landscape* — $L(\theta)$ drawn as terrain, whose ground coordinates are the parameters and whose height at each point is the loss there. The next section will describe training as a walk downhill on this terrain; here the terrain itself is constructed, first for one parameter, then for two.

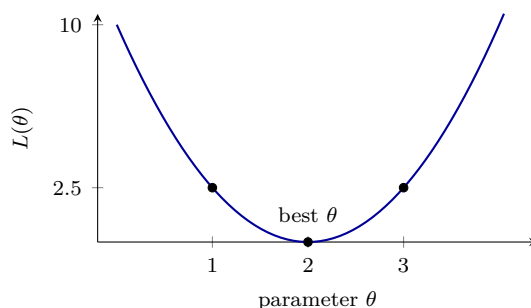
A *landscape* computed by hand. Take the smallest possible model, one parameter:

$$\hat{y} = f_{\theta}(x) = \theta x,$$

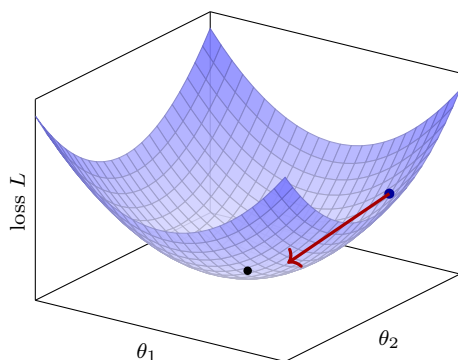
and two training examples, $(x_1, y_1) = (1, 2)$ and $(x_2, y_2) = (2, 4)$. Start from the training-loss formula of the setup, with the MSE as ℓ , and populate the numbers step by step:

$$\begin{aligned} L(\theta) &= \frac{1}{2} \sum_{n=1}^2 (\hat{y}_n - y_n)^2 = \frac{1}{2} [(\theta x_1 - y_1)^2 + (\theta x_2 - y_2)^2] && (N = 2; \hat{y}_n = \theta x_n) \\ &= \frac{1}{2} [(\theta \cdot 1 - 2)^2 + (\theta \cdot 2 - 4)^2] && \text{(insert the two examples)} \\ &= \frac{1}{2} [(\theta - 2)^2 + 4(\theta - 2)^2] && \text{(factor } (2\theta - 4)^2 = 4(\theta - 2)^2) \\ &= \frac{5}{2} (\theta - 2)^2. \end{aligned}$$

This $L(\theta)$ is the model's entire landscape — one axis, the parameter θ ; the height at each point, the loss there:

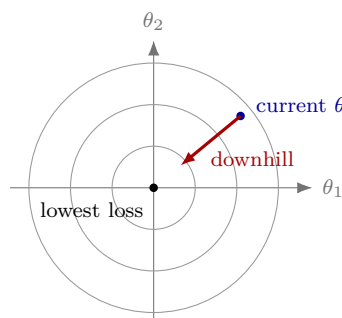


Two parameters. With $\theta = (\theta_1, \theta_2)$, the landscape becomes a *surface* over the plane of parameter settings — shown below in the two standard views. The *same* situation is marked in both: the blue dot is the current parameter setting, sitting at some height (its loss); the red arrow is the downhill direction toward the bottom. On the right, each ring is a *contour line*: a curve along which the loss keeps one constant value.



the surface: height = loss $L(\theta_1, \theta_2)$;

the dot's height is its loss, the arrow walks downhill



the same surface from above: contour lines

Every training run is a walk on this landscape: the current parameters are one point, the loss is the height there, and learning means moving toward lower ground. Real models have millions of parameters — a landscape with millions of axes, not drawable, and far less regular than a bowl — but the question it poses survives unchanged: *which direction is downhill, and how large a step*

to take? The gradient answers the first half; the next section — gradient descent — answers the second.

2.5 Symbols

New persistent symbols: θ , $L(\theta)$, x , y , $\hat{y}$ — see the [list of symbols](#). Local to this section:

Local symbol	Meaning (this section)
f_θ	the model: a function whose behavior is set by θ
$\ell(\hat{y}, y)$	one example's loss (the dataset loss L averages these)
N, n	number of training examples; the index over them
q_{true}	the probability the model assigned to the correct class
K	the number of classes
contour line	a curve on the landscape along which the loss is constant

3 Gradient descent

This section gives the algorithm that walks on §2’s landscape: *gradient descent* — find which direction is downhill, take a small step that way, repeat. Every neural network in use today is trained by some variant of this one procedure.

3.1 The update rule

Two ingredients:

- The gradient ∇L is the vector of all partial derivatives $\partial L / \partial \theta_i$. It points in the direction of steepest *increase* of the loss; its negative, $-\nabla L$, points in the direction of steepest *decrease*.
- The *learning rate* η is a small positive number: the step size. (It gets its own section, §4; for now, treat it as a fixed small constant such as 0.1.)

Why “steepest”. Take a small step δ (a vector: how much each parameter moves). For small steps the loss responds linearly,

$$L(\theta + \delta) \approx L(\theta) + \nabla L(\theta) \cdot \delta,$$

so the change in loss is the *dot product* of the gradient with the step (dot product: multiply matching entries, then add). A dot product is largest when the two vectors point the same way and most negative when they point opposite ways — so among all steps of equal length, the step *along* ∇L raises the loss fastest, and the step *against* it lowers the loss fastest. That is the entire content of “points in the direction of steepest increase”.

Concretely, take two parameters and $\nabla L = (3, 4)^\top$, and compare steps of length 1:

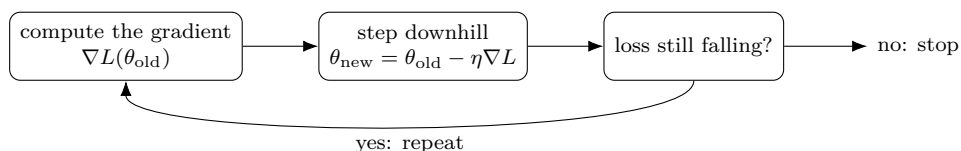
Step δ (length 1)	Change $\nabla L \cdot \delta$
along the gradient, $(0.6, 0.8)^\top$	$3 \cdot 0.6 + 4 \cdot 0.8 = 5$ the largest possible
along θ_1 only, $(1, 0)^\top$	3
along θ_2 only, $(0, 1)^\top$	4
against the gradient, $(-0.6, -0.8)^\top$	-5 the most negative

No unit step does better than ± 5 — and 5 is exactly the gradient’s length, $\|\nabla L\| = \sqrt{3^2 + 4^2}$: the direction of ∇L is the steepest one, and its length is that steepest rate.

One step of gradient descent moves the parameters a distance η along the downhill direction:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla L(\theta_{\text{old}}).$$

Training is this step in a loop:



3.2 A descent computed by hand

Take §2’s one-parameter landscape, $L(\theta) = \frac{5}{2}(\theta - 2)^2$. Its derivative, computed step by step — the constant factor $\frac{5}{2}$ stays, the square differentiates by the power rule, and the inner $(\theta - 2)$ contributes

derivative 1 by the chain rule:

$$L'(\theta) = \frac{5}{2} \cdot \underbrace{2(\theta - 2)}_{\text{power rule}} \cdot \underbrace{1}_{\text{inner derivative}} = 5(\theta - 2).$$

In one dimension the gradient is simply this derivative, and a direction is simply a sign:

- $L' > 0$: the loss increases to the *right* of θ — formally, $L(\theta + h) > L(\theta) > L(\theta - h)$ for small $h > 0$, an ordering of values;
- $L' < 0$: the loss increases to the *left* — the same ordering, mirrored.

Left and right are movement along the θ -axis, the only thing training can change; the increase is what the loss value does in response. The magnitude of L' says how steep the rise is.

Now run the algorithm on this landscape. Choose $\eta = 0.3$, deliberately a little large so that one run will show both signs of L' , and start at $\theta_0 = 0$. Steps are counted $t = 0, 1, 2, \dots$; the count is 0-based because θ_0 is the starting value before any step. Populating the update rule with these numbers:

$$\theta_{t+1} = \theta_t - \eta L'(\theta_t) = \theta_t - 1.5(\theta_t - 2),$$

so, starting from $\theta_0 = 0$:

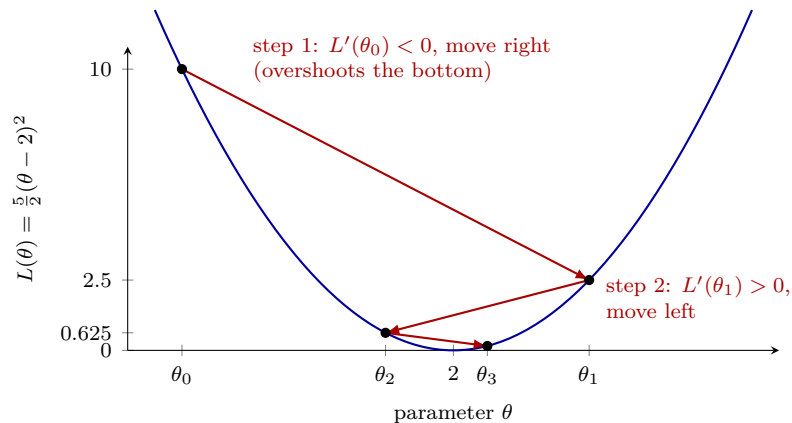
t	θ_t	$L'(\theta_t) = 5(\theta_t - 2)$	step $-\eta L'(\theta_t)$	direction	$L(\theta_t)$
0	0	-10	+3	right	10
1	3	+5	-1.5	left	2.5
2	1.5	-2.5	+0.75	right	0.625
3	2.25	+1.25	-0.375	left	0.156
4	1.875	-0.625	+0.1875	right	0.039

Both signs of L' appear inside this single run, and each step's direction follows by the same chain of small links:

- At $\theta_0 = 0$: the derivative is negative, $L'(\theta_0) = -10 < 0$. Its negation is therefore positive, and so is the step, $-\eta L'(\theta_0) = 0.3 \cdot 10 = +3$. A positive step is movement in the $+\theta$ direction: to the *right*. This step *overshoots* the bottom and lands at $\theta_1 = 3$, on the other side.
- At $\theta_1 = 3$: the derivative is positive, $L'(\theta_1) = +5 > 0$, so the step $-\eta L'(\theta_1) = -1.5$ is negative. A negative step is movement in the $-\theta$ direction: to the *left*.

Every crossing flips the sign of L' , and the sign flips the step's direction: the walk zigzags across the bottom. Each step halves the distance to the bottom at $\theta = 2$, $\theta_{t+1} - 2 = -0.5(\theta_t - 2)$ — the minus sign is the crossing — and the loss falls by a factor of four.

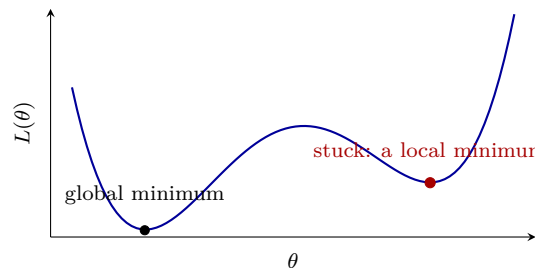
The walk on this same landscape, arrows in the table's order:



Note what nobody had to tell the algorithm: the steps shrink on their own. Near the bottom the ground is flat, the gradient is small, and $\eta \nabla L$ is therefore a short step — gradient descent slows down exactly where care is needed.

3.3 Where the walk ends

The walk stops where $\nabla L = 0$: flat ground, zero update. On a bowl that is the single lowest point; real landscapes have more kinds of flat:



A *local minimum* is the bottom of some valley; the deepest point overall is the *global minimum*. A walk that descends into the shallower valley stays there — every direction out is uphill. In practice this is accepted: trained models almost never sit at the global minimum, and it barely matters. In high dimension most flat points are *saddle points* (flat along some axes, downhill along others), so escapes usually exist; mini-batch noise (§5) shakes the walk out of shallow traps; and the global minimum of the training loss is not even the goal — reaching it usually means memorizing the training set, the overfitting of §6.

3.4 Why this simple method

Gradient descent is old — it goes back to Cauchy (1847) — and deliberately simple. Its natural upgrades exist, but each is priced out at the scale of modern models.

The best-known relative is *Newton's method*, which divides by the second derivative instead of using a fixed step size: $\theta_{t+1} = \theta_t - L'(\theta_t)/L''(\theta_t)$. The curvature then chooses the step, and on the parabola above ($L'' = 5$) it lands exactly at the bottom in a single step: $\theta - 5(\theta - 2)/5 = 2$. The price is memory: for a model with millions of parameters the second derivative is a millions-by-millions matrix (the *Hessian*), while the gradient is only a vector. Newton's precision does not fit; the cheap first-derivative step does.

The second price is time, and gradient descent itself still pays it: computing ∇L exactly means evaluating the gradient over *all* N training examples for every single step — unaffordable when N is in the billions. The fix, §5, is to estimate the gradient from a small random batch.

3.5 Symbols

New persistent symbols: ∇L and η — see the [list of symbols](#). Local to this section:

Local symbol	Meaning (this section)
$\theta_{\text{old}}, \theta_{\text{new}}$	the parameters before and after one update step
θ_t	the parameters after t steps ($t = 0, 1, 2, \dots$; $\theta_0 = \text{start}$)
$L'(\theta)$	the one-parameter landscape's derivative (its slope)
local minimum	the bottom of some valley: flat, but not necessarily the deepest
saddle point	flat along some axes, still downhill along others
global minimum	the lowest point of the whole landscape

4 The learning rate

This section studies the choice that §3 left open. The *learning rate* η is the positive number in the update rule that scales the gradient into the actual step. This one number decides whether gradient descent creeps, converges, or explodes — the section shows all of these behaviors on one landscape, locates the exact boundaries between them, and then shows how a working value is found in practice.

4.1 What the learning rate controls

Recall the update rule:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla L(\theta_{\text{old}}).$$

The gradient supplies the direction and grows or shrinks with the slope; η converts it into a displacement. The step's length is $\eta \|\nabla L(\theta_{\text{old}})\|$: doubling η doubles how far every step travels.

Why the step must be kept small: the gradient is measured at θ_{old} , and the linear picture behind it, $L(\theta + \delta) \approx L(\theta) + \nabla L(\theta) \cdot \delta$, is accurate only *near* that point. A short step lands where the picture predicted; a long step lands on ground the measurement said nothing about — possibly across the valley, possibly higher up the other wall. The learning rate is the trust placed in one local measurement.

Note also what the update rule does *not* do: nothing in the loop adjusts η . The person running the training chooses it before the loop starts. A setting of this kind is called a *hyperparameter*: a training setting chosen by the person, not learned by the training. The parameters θ are learned; the learning rate — and later the batch size (§5) and the model's size — are hyperparameters.

4.2 One landscape, six learning rates

Every possible behavior of the learning rate can be shown on §3's landscape, $L(\theta) = \frac{5}{2}(\theta - 2)^2$ with derivative $L'(\theta) = 5(\theta - 2)$, starting at $\theta_0 = 0$ (step count $t = 0, 1, 2, \dots$ as in §3: 0-based, θ_0 the start). Populate the update rule, keeping η symbolic this time:

$$\theta_{t+1} = \theta_t - \eta L'(\theta_t) = \theta_t - 5\eta(\theta_t - 2).$$

The landscape's minimum sits at $\theta = 2$, and θ_t is the walk's position after t steps — so their difference $\theta_t - 2$, position minus minimum, is the walk's current *signed distance* to the minimum: negative means the walk stands left of it, positive means right of it. At the start, $\theta_0 - 2 = 0 - 2 = -2$: two units to the left. How does one step change this distance? Subtract 2 from both sides of the update rule, then factor out $(\theta_t - 2)$:

$$\underbrace{\theta_{t+1} - 2}_{\text{new distance}} = \underbrace{(\theta_t - 2)}_{\text{old distance}} - 5\eta(\theta_t - 2) = (1 - 5\eta)(\theta_t - 2).$$

Every step multiplies the distance by the same fixed number, the *step factor*

$$r = 1 - 5\eta,$$

and after t steps $\theta_t - 2 = r^t(\theta_0 - 2)$. The entire effect of the learning rate on this landscape is packed into r :

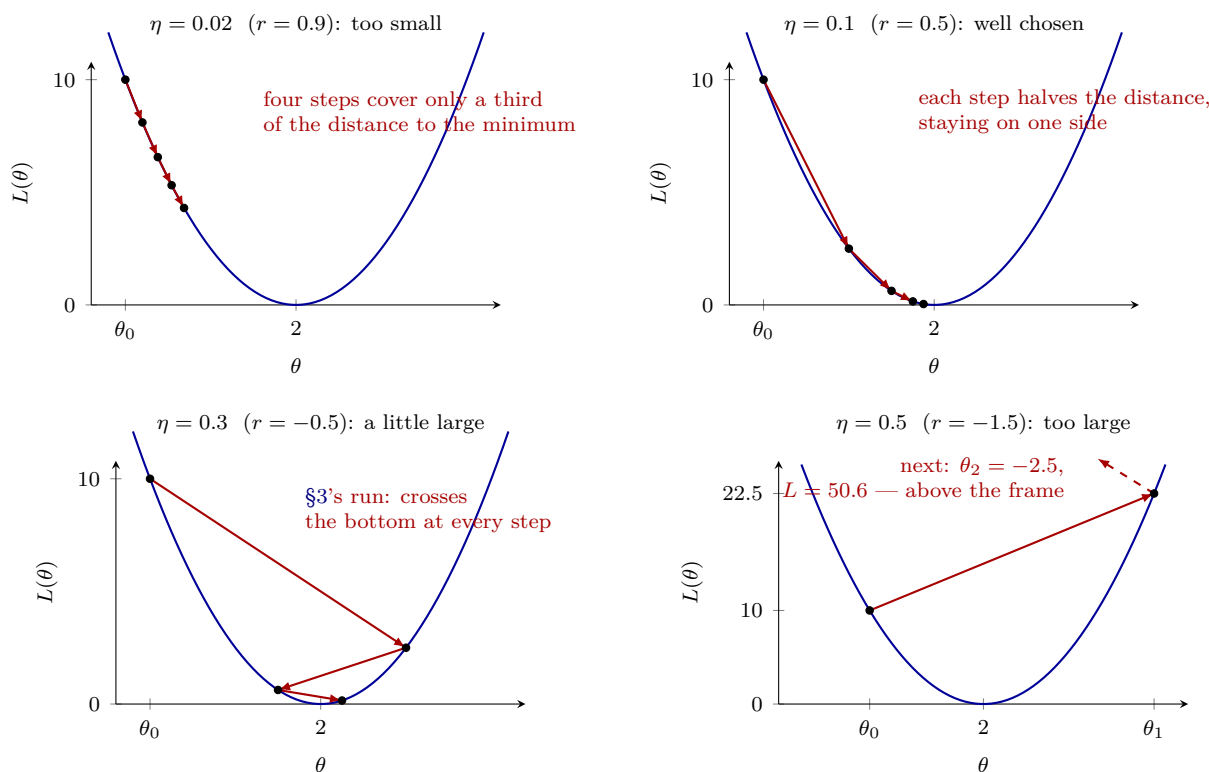
- $|r| < 1$: the distance shrinks at every step — the walk *converges* to the minimum;

- $|r| > 1$: the distance grows at every step — the walk *diverges*;
- $r < 0$: the sign of $\theta_t - 2$ flips at every step — the walk crosses the bottom each time.

Populated with six concrete learning rates:

η	$r = 1 - 5\eta$	$\theta_0, \dots, \theta_4$	behavior
0.02	0.9	0, 0.2, 0.38, 0.542, 0.6878	converges — slowly, on one side
0.1	0.5	0, 1, 1.5, 1.75, 1.875	converges — quickly, on one side
0.2	0	0, 2, 2, 2, 2	reaches the minimum in one step
0.3	-0.5	0, 3, 1.5, 2.25, 1.875	converges — crossing sides (§3's run)
0.4	-1	0, 4, 0, 4, 0	never settles: jumps between two points
0.5	-1.5	0, 5, -2.5, 8.75, -8.125	diverges — every step lands farther out

Four of these runs, drawn on the landscape:



four runs on the same landscape $L(\theta) = \frac{5}{2}(\theta - 2)^2$, all starting at $\theta_0 = 0$

Note that with $\eta = 0.5$ the loss *risks from the very first step*: $L(\theta_0) = 10$ but $L(\theta_1) = 22.5$. The step overshoots the bottom so far that it lands higher on the opposite wall, the gradient there is even larger, so the next step is even longer — each step feeds the next, and the walk leaves the valley.

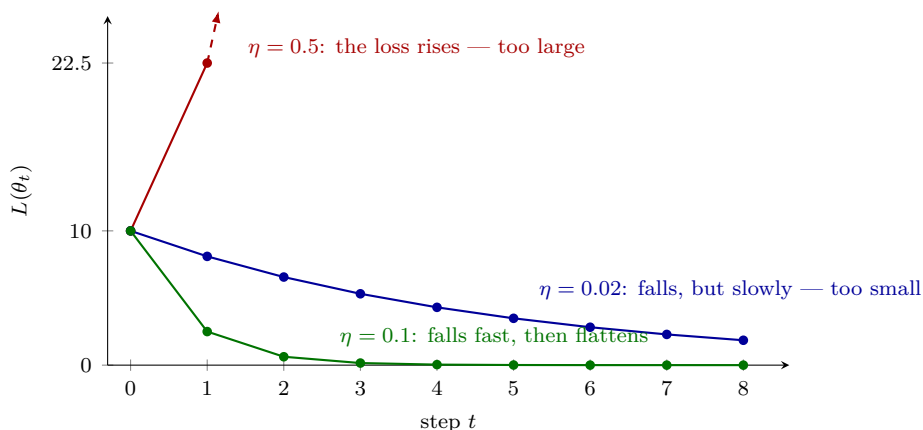
Two of the table's rows are exact boundaries. At $\eta = 0.2$ the step factor is $r = 0$: the walk lands exactly at the minimum in one step. This is no accident — $0.2 = 1/L''$ (the curvature here is $L'' = 5$), and at that value the gradient step $-\eta L' = -L'/L''$ coincides with Newton's step from §3. At $\eta = 0.4$ the step factor is $r = -1$: each step exactly mirrors the point to the other side ($\theta = 0 \leftrightarrow \theta = 4$, both at loss 10), forever. Below 0.4 the walk converges; above it, it diverges.

Both special values are set by the curvature: for a parabola with second derivative L'' the same

computation gives $r = 1 - \eta L''$, so the one-step value is $\eta = 1/L''$ and the divergence boundary is $\eta = 2/L''$. A steeper valley — larger L'' — tolerates only smaller learning rates. And here is the practical problem: a real landscape bends by different amounts in different places and directions, and its curvature is exactly the millions-by-millions object that §3 showed cannot be computed. No formula supplies the right η . It is found by watching the training run.

4.3 Reading the loss curve

A real landscape cannot be drawn — it has millions of axes — but one plot is always available: the *loss curve*, the loss value plotted against the step count. It is the standard instrument for judging the learning rate, because the regimes of the table have three recognizable shapes:



the loss per step for three of the table's runs on $L(\theta) = \frac{5}{2}(\theta - 2)^2$, starting at $\theta_0 = 0$

(The $\eta = 0.3$ run is not drawn because its loss curve coincides with the $\eta = 0.1$ one: $|-0.5| = |0.5|$, and the loss reports the distance to the minimum regardless of which side the walk is on.)

For a real model the curves are not these exact exponentials, but the three shapes are the same, and they are diagnosed the same way: a loss that rises or swings wildly means the learning rate is too large — in code it soon overflows to **NaN** (“not a number”); a loss that falls steadily but very slowly means it is too small; a loss that falls fast and then flattens means the value is workable. The standard search reflects how wide the range of candidates is: try values spaced by factors of ten (for neural networks typically 10^{-4} to 10^{-1}), run each briefly, and keep the largest η whose curve still falls cleanly — the largest, because it makes the most progress per step.

One refinement is worth knowing about now: in real training η is usually not held fixed. It is set larger at the start, when the parameters are far from good and long steps are safe, and lowered as training proceeds, when the walk is near a minimum and the $\eta = 0.5$ behavior must be avoided. A plan of this kind is called a *learning-rate schedule*; the chapter on neural-network mechanics covers the common ones.

4.4 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$\theta_t - 2$	the signed distance from the walk's position to the minimum at $\theta = 2$
$r = 1 - 5\eta$	the step factor: each step multiplies the distance to the minimum by it
hyperparameter	a training setting chosen by the person, not learned by the training
loss curve	the plot of the loss value against the step count
learning-rate schedule	a plan that changes η over the course of training
L''	the second derivative: the curvature of the landscape

5 Stochastic gradient descent

Gradient descent, as described so far, reads all N training examples to make a *single* update step. This section replaces it with the version every large model actually uses: *stochastic gradient descent* (SGD), which reads only a small random sample — a *batch* — per step. The point of the change is what a pass through the data then buys. Progress is measured in update steps, not in examples read: reading all N examples once gives full-batch gradient descent *one* step, while the same reading, cut into batches of B examples, gives SGD N/B steps. Each SGD step follows a noisy estimate of the gradient rather than the exact one — but many roughly-right steps cross the landscape faster than one perfect step. The section builds the estimate, computes single steps and whole passes by hand, and shows what the noise does near a minimum.

5.1 The cost problem

§2 built the training loss as an average over the N examples:

$$L(\theta) = \frac{1}{N} \sum_{n=1}^N \ell(\hat{y}_n, y_n).$$

Differentiation distributes over a sum and passes through the constant $1/N$, so the gradient is the average of the *per-example* gradients:

$$\nabla L(\theta) = \frac{1}{N} \sum_{n=1}^N \nabla \ell_n(\theta), \quad \ell_n(\theta) = \ell(f_\theta(x_n), y_n)$$

— example n 's loss, viewed as a function of the parameters.

Check this on the running dataset of §2: the model $\hat{y} = \theta x$ with examples $(x_1, y_1) = (1, 2)$ and $(x_2, y_2) = (2, 4)$. Each example's loss and its derivative (power rule, then the inner derivative — here x_n — by the chain rule):

$$\begin{aligned} \ell_1(\theta) &= (\theta \cdot 1 - 2)^2, & \ell'_1(\theta) &= 2(\theta - 2) \cdot 1 = 2(\theta - 2); \\ \ell_2(\theta) &= (\theta \cdot 2 - 4)^2, & \ell'_2(\theta) &= 2(2\theta - 4) \cdot 2 = 8(\theta - 2). \end{aligned}$$

Populating the average formula with the two derivatives (in one dimension the gradient is simply the derivative L'):

$$\begin{aligned} L'(\theta) &= \frac{1}{2} [\ell'_1(\theta) + \ell'_2(\theta)] && \text{(the average formula, } N = 2) \\ &= \frac{1}{2} [2(\theta - 2) + 8(\theta - 2)] && \text{(insert the two derivatives)} \\ &= \frac{1}{2} \cdot 10(\theta - 2) = 5(\theta - 2) && \text{(add, then halve)} \end{aligned}$$

— exactly the $L'(\theta) = 5(\theta - 2)$ that §3 computed from the landscape formula $L(\theta) = \frac{5}{2}(\theta - 2)^2$. The two routes agree: average the per-example derivatives, or differentiate the landscape directly.

For $N = 2$ this average costs nothing. For a modern dataset, look at what it buys: with $N = 10^9$, one update step costs a billion per-example gradient computations — a billion reads for a *single move* of the parameters, when a training run needs on the order of 10^5 moves. The reading is also mostly wasted, because the gradient is an *average*, and an average does not need every member of the population: to estimate the average height of a million people, measuring a randomly chosen hundred is enough — a poll works this way. An estimate from a hundred examples is not exact, but it is cheap enough to afford a move per hundred reads instead of a move per billion. Building that estimate is the next subsection.

5.2 The fix: average over a random sample

A *mini-batch* is a small random subset of the training examples, drawn fresh at every step. Two symbols are needed, one for *which* examples and one for *how many*: the calligraphic $\mathcal{B}$ is the batch itself — the set of drawn indices — and the italic B (the *batch size*) is the number of elements in it, a hyperparameter in §4’s sense. Keep the two shapes apart: in this document B is always a count, never the batch. (The warning is worth stating because much ML writing uses the plain letter B for the batch itself; here that role belongs exclusively to $\mathcal{B}$.) The *batch gradient* is the same average as before, taken over the sample only:

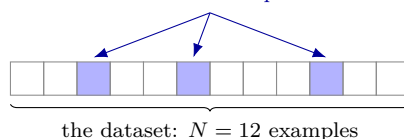
$$\nabla L_{\mathcal{B}}(\theta) = \frac{1}{B} \sum_{n \in \mathcal{B}} \nabla \ell_n(\theta),$$

and one step of *stochastic gradient descent* is the update rule with this estimate in the exact gradient’s place:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla L_{\mathcal{B}}(\theta_{\text{old}}).$$

(*Stochastic* is the technical word for “random”. Historically the name meant $B = 1$; in modern usage “SGD” means this mini-batch version, with B typically between 32 and a few thousand.)

the mini-batch $\mathcal{B}$ — $B = 3$ examples drawn at random



The estimate has one property bought and one property that makes the purchase safe.

Bought: the cost. One step now computes B per-example gradients instead of N . With $B = 100$ and $N = 10^9$, each step is ten million times cheaper — or, counted the other way, the reading that bought full-batch descent *one* move now buys SGD ten million moves.

Safe: correct on average. Every example is equally likely to be drawn, so no part of the dataset is favored: averaged over all possible batches, the batch gradient equals the exact gradient (the technical term: the estimate is *unbiased*). On the running dataset with $B = 1$ there are exactly two possible batches — drawing example 1 gives the estimate $2(\theta - 2)$, drawing example 2 gives $8(\theta - 2)$, and their average is $5(\theta - 2)$, the exact gradient. But any *single* batch misses: $2(\theta - 2)$ is too weak, $8(\theta - 2)$ too strong. And which miss occurs depends on the draw — random, different at every step. This random miss is called the *noise* in the estimate; it is demonstrated at work in *The noise, shown by hand* below. Do not confuse it with *measurement* noise — error in the data values themselves. The estimate’s noise comes from *sampling*, reading only B of the N examples, and exists even when every data point is measured perfectly: the two examples here lie exactly on $y = 2x$, and the estimates miss anyway.

5.3 One step in full: what is read, what is ignored

The previous subsection defined SGD by two formulas — the batch gradient (draw B examples, average their gradients) and the update rule that steps with the result:

$$\nabla L_{\mathcal{B}}(\theta) = \frac{1}{B} \sum_{n \in \mathcal{B}} \nabla \ell_n(\theta), \quad \theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla L_{\mathcal{B}}(\theta_{\text{old}}).$$

This subsection performs one such update exactly, on a dataset just big enough that the ignoring becomes visible. Keep the model $\hat{y} = \theta x$, and extend the running pair — examples 1 and 2 below are (1, 2) and (2, 4) — to six examples:

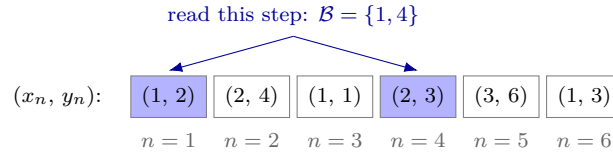
n	1	2	3	4	5	6
x_n	1	2	1	2	3	1
y_n	2	4	1	3	6	3

Each example's loss and derivative follow the same differentiation as before, now for a general pair (x_n, y_n) :

$$\ell_n(\theta) = (\theta x_n - y_n)^2, \quad \ell'_n(\theta) = 2(\theta x_n - y_n) \cdot x_n$$

(power rule, then the inner derivative x_n). Let the current position be $\theta_{\text{old}} = 1$, with $B = 2$ and $\eta = 0.1$. One step of SGD, in its three moves:

1. *Draw*. The random draw comes out $\mathcal{B} = \{1, 4\}$: this step will read examples 1 and 4 and no others.



the four unshaded examples are not computed at all in this step

2. *Compute only what was drawn*. Populate ℓ'_n for the two drawn examples at $\theta = 1$:

$$\ell'_1(1) = 2(1 \cdot 1 - 2) \cdot 1 = -2, \quad \ell'_4(1) = 2(1 \cdot 2 - 3) \cdot 2 = -4.$$

Examples 2, 3, 5, 6 are not run through the model, and their derivatives are never formed — four of the six examples play no part in this step.

3. *Average, then step*.

$$\nabla L_{\mathcal{B}}(1) = \frac{1}{2}[(-2) + (-4)] = -3, \quad \theta_{\text{new}} = 1 - 0.1 \cdot (-3) = 1.3.$$

For contrast, the exact gradient at $\theta = 1$ evaluates all six examples — the derivatives are $-2, -8, 0, -4, -18, -4$ — and averages them to -6 . The estimate -3 points the same way with a different size, at a third of the cost (2 evaluations instead of 6).

The ignoring is per step, not permanent: the next steps draw fresh batches. Continuing with the draws $\{3, 6\}$ and then $\{2, 5\}$:

t	θ_t	drawn $\mathcal{B}$	$\ell'_n(\theta_t)$, drawn only	$\nabla L_{\mathcal{B}}$	step	θ_{t+1}
0	1	$\{1, 4\}$	$-2, -4$	-3	$+0.3$	1.3
1	1.3	$\{3, 6\}$	$+0.6, -3.4$	-1.4	$+0.14$	1.44
2	1.44	$\{2, 5\}$	$-4.48, -10.08$	-7.28	$+0.728$	2.168

Each step read two examples and ignored four — yet across the three steps every example was read exactly once. This is the intended reading of “batching”, at its three time scales:

- one *step* reads only the B drawn examples;
- one complete pass reads all N of them, a batch at a time (such a pass gets a name in the practice subsection below);

- a whole training run repeats such passes many times, so every example is read many times over.

Batching never shrinks the training data from N to B — it only sets how many examples a single update step reads. At real scale the proportions are extreme: with $N = 10^9$ and $B = 100$, each step reads 100 examples and ignores 999,999,900 — and training still reaches all of them, a batch at a time.

Note also $t = 1$ in the table: the two drawn gradients disagree in sign, because example 3's own best value ($\theta = 1$) is already behind the walk while example 6's ($\theta = 3$) is still ahead. The next subsection turns that disagreement into its subject.

5.4 The noise, shown by hand

This subsection makes the noise visible. It builds the smallest dataset whose examples *disagree*, runs SGD on it, and watches what happens near the minimum: the walk stops converging and starts bouncing. Why that happens, how large the bounce is, and what cures it are all computable by hand here.

A dataset that disagrees. The running pair cannot show the noise, because its two examples agree: $\theta = 2$ fits both exactly ($2 \cdot 1 = 2$, $2 \cdot 2 = 4$), so at the minimum both per-example gradients are zero at once — every batch, however drawn, is satisfied there. Real datasets never agree like this, for several reasons at once: measurements carry error; reality has more influences than the model's input captures, so equal inputs can arrive with different targets; and some entries are outright contaminated — mistyped, mislabeled, or corrupted. Disagreement is therefore the normal condition of data, not the mark of a broken dataset. To create a disagreement, change one number: keep $(1, 2)$, but replace $(2, 4)$ by $(2, 3)$. The new point is chosen *off the pattern* on purpose: $(1, 2)$ lies exactly on the line $y = 2x$, and $(2, 3)$ does not — it plays the role of a real measurement, which never lands exactly on the model's ideal curve. Nothing marks it as “the wrong point” from the model's side: the model only sees two examples that no single θ fits, with no way to tell which one is off — in real data, usually both are, a little. Example 1 is still fit exactly by $\theta = 2$ (from $\theta \cdot 1 = 2$); example 2 is now fit exactly by $\theta = 1.5$ (from $\theta \cdot 2 = 3$). No θ fits both.

The landscape of the disagreeing dataset. Computed as in §2:

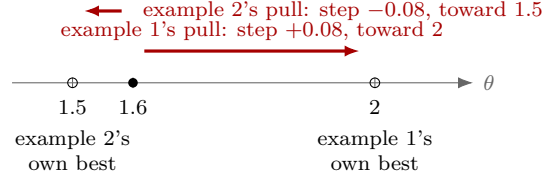
$$\begin{aligned} L(\theta) &= \frac{1}{2} \left[(\theta - 2)^2 + (2\theta - 3)^2 \right] && \text{(MSE over the changed dataset)} \\ &= \frac{1}{2} \left[5\theta^2 - 16\theta + 13 \right] && \text{(expand both squares, add)} \\ &= \frac{5}{2} (\theta - 1.6)^2 + 0.1 && \text{(rewrite; expanding it back gives the line above).} \end{aligned}$$

Still a parabola with the same curvature 5, but two things changed, both caused by the disagreement. The minimum moved to $\theta = 1.6$ — a compromise between the examples' own best values 2 and 1.5. And the loss *at* the minimum is now 0.1, not 0: no parameter value fits both examples, so even the best one keeps some loss.

The pulls at the minimum. Example 2's loss became $\ell_2(\theta) = (2\theta - 3)^2$, so $\ell'_2(\theta) = 2(2\theta - 3) \cdot 2 = 8(\theta - 1.5)$; example 1's is unchanged, $\ell'_1(\theta) = 2(\theta - 2)$. Populate both at the minimum $\theta = 1.6$:

$$\ell'_1(1.6) = 2(1.6 - 2) = -0.8, \quad \ell'_2(1.6) = 8(1.6 - 1.5) = +0.8.$$

The exact gradient — their average — is zero there, but not because each example is satisfied: the two pulls cancel. With $\eta = 0.1$, a step on example 1 alone is $-\eta \ell'_1(1.6) = +0.08$, toward 2; a step on example 2 alone is -0.08 , toward 1.5:



What SGD does with the disagreement. The exact gradient averages the two pulls away. A batch of size 1 does not: it obeys the drawn example alone, stepping toward *that example's* own best value — never toward 1.6 itself. The steps follow the same algebra as §4's step factor, applied per example (with $\eta = 0.1$):

$$\text{drawing example 1: } \theta_{t+1} - 2 = (1 - 0.1 \cdot 2)(\theta_t - 2) = 0.8(\theta_t - 2),$$

$$\text{drawing example 2: } \theta_{t+1} - 1.5 = (1 - 0.1 \cdot 8)(\theta_t - 1.5) = 0.2(\theta_t - 1.5).$$

In words: a step on example 1 shrinks the walk's gap to 2 to 80% of what it was; a step on example 2 shrinks the gap to 1.5 to 20%.

The run. Take $B = 1$, $\eta = 0.1$, $\theta_0 = 0$, and draw the examples in the fixed alternation 1, 2, 1, 2, ... (a real run draws at random; the fixed order keeps the table checkable and shows the same behavior):

t	θ_t	drawn	$\ell'(\theta_t)$	step $-\eta \ell'(\theta_t)$	θ_{t+1}
0	0	1	-4	+0.4	0.4
1	0.4	2	-8.8	+0.88	1.28
2	1.28	1	-1.44	+0.144	1.424
3	1.424	2	-0.608	+0.0608	1.4848
4	1.4848	1	-1.0304	+0.10304	1.58784
5	1.58784	2	+0.70272	-0.070272	1.517568
6	1.517568	1	-0.964864	+0.0964864	1.6140544

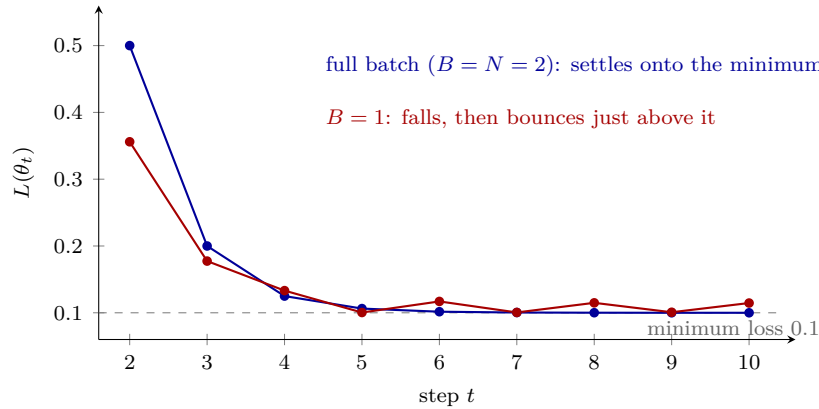
Phase 1: far from the minimum ($t = 0$ to 4). The noise hardly matters here. The walk stands left of both 1.5 and 2, so both examples pull right, and the drawn example changes only the step's *size*. This is why SGD crosses open ground about as well as exact gradient descent does.

Phase 2: near the minimum (from $t = 5$). The walk now stands between 1.5 and 2, the pulls conflict, and each step drags the walk past 1.6 toward the drawn example's own side. Continue the alternation and the positions approach a fixed two-step cycle (values rounded to four decimals):

$$2 + 0.8(1.5238 - 2) = 1.6190, \quad 1.5 + 0.2(1.6190 - 1.5) = 1.5238$$

— example 1 sends 1.5238 to 1.6190, example 2 sends it back. The walk bounces between these two points forever, and it *cannot* rest at 1.6: a batch gradient there is $\ell'_1(1.6) = -0.8$ or $\ell'_2(1.6) = +0.8$, never 0. Only the average over both examples vanishes at the minimum, and a batch of one never takes that average. With noisy gradients, “zero exact gradient” no longer means “zero step”.

The loss curve. The two phases, seen through the loss:



loss per step on the disagreeing dataset ($\eta = 0.1$, both runs from $\theta_0 = 0$), drawn from step 2 so the floor is visible

The full-batch curve settles onto the minimum loss 0.1 and stays. The $B = 1$ curve falls just as fast, then *stops falling*: it keeps popping back up to ≈ 0.115 , forever. The loss level just above the minimum that the bouncing cannot descend below is called the *noise floor*. This is the general picture of SGD with a fixed learning rate: it converges not to the minimum but to a small region around it, and keeps moving inside that region.

The cure: shrink η . The bounce’s size is set by the learning rate. Halve it to $\eta = 0.05$ and the cycle tightens from (1.5238, 1.6190) to (1.5652, 1.6087); shrink η further and the bounce narrows toward the minimum. This is a second, independent reason why the learning-rate schedules of §4 end small: a large η early crosses the landscape quickly, and a small η late lowers the noise floor.

Reading real loss curves. An SGD loss curve is jagged — it rises on individual steps even when training is healthy, exactly as the red curve above does. Judging a learning rate by §4’s three shapes therefore means judging the curve’s *trend*, not its individual wiggles.

The noise also helps. On the many-valley landscapes of §3, a walk sitting in a shallow local minimum can be knocked over the rim by a batch whose examples pull elsewhere — the exact gradient, zero there, would hold it in place forever. A second benefit is empirical: models trained with noisy gradients often generalize better than models driven by the exact gradient; §6 returns to this.

5.5 Epochs and practice

The epoch. An *epoch* is one complete pass through the training data: shuffle the examples’ order, cut it into N/B consecutive batches of size B , consume one batch per step — N/B steps in all (B counts a batch’s size; N/B counts the batches). Reshuffling each epoch uses every example exactly once per epoch, where independent draws would miss some by chance. For the six-example dataset with $B = 2$, the shuffle 1, 4, 3, 6, 2, 5 gives the batches $\{1, 4\}$, $\{3, 6\}$, $\{2, 5\}$ of the run above; with $N = 50,000$ and $B = 100$, an epoch is 500 steps. Training lengths are stated in epochs.

What an epoch buys. An epoch of SGD reads exactly as many examples as *one* step of full-batch gradient descent — all N — but turns the reading into N/B updates instead of one. In the six-example run above: three moves ($1 \rightarrow 1.3 \rightarrow 1.44 \rightarrow 2.168$) against one move ($1 - 0.1 \cdot (-6) = 1.6$), for the same six examples read. The endpoints differ — equal is only the price. Neither run has reached the minimum, which sits at $\theta = 1.9$ (where the exact gradient vanishes: $20\theta = 38$): full-batch stops 0.3 before it, SGD ends 0.268 past it. What SGD wins is chances to steer — a gradient

is stale one step after it is measured (§4), so spending all N examples to perfect a direction that the next step discards is waste.

No two runs repeat. “Correct on average” holds at a fixed position; once a step is taken, the walks stand at different points and measure different gradients, so batched and full-batch descent trace different paths — and so do two SGD runs with different shuffles. Had the epoch above started with $\{2, 5\}$, the first step would be $1 - 0.1 \cdot (-13) = 2.3$ instead of 1.3. Two trainings with different shuffles therefore end at different final weights, of roughly equal quality — acceptable, because the goal was never one exact endpoint: every such walk drifts into the same noise-floor region around the minimum, and that region is what training relies on.

Choosing B . The batch size is pushed from both directions:

- *Too small wastes the hardware.* A GPU computes a whole batch’s per-example gradients simultaneously, so a batch of 256 takes about as long as a batch of one — at $B = 1$ an epoch takes roughly 256 times longer for the same reading.
- *Too large wastes computation.* The estimate is already roughly right at modest B , and further precision is bought at a diminishing rate: the typical miss shrinks like $1/\sqrt{B}$ (a standard fact about averages of random draws), so the batch must *quadruple* to halve the noise. Meanwhile each step costs proportionally more, and the noise removed was even useful (the previous subsection).

What the noise grading means. The table’s *Gradient* column grades how badly each variant’s guess misses the exact gradient; it says nothing about time (time is the cost column). More examples in the average means more too-strong and too-weak pulls canceling, so the miss shrinks as B grows: at $B = N$ the guess *is* the exact gradient (no miss); at $B = 1$ one example’s pull stands alone (the biggest misses). On the running pair: $B = 1$ guesses $2(\theta - 2)$ or $8(\theta - 2)$, each missing $5(\theta - 2)$ by $3(\theta - 2)$; $B = N = 2$ misses by nothing. Least noisy is therefore not best: $B = N$ wins this column and loses the one that matters more — updates per reading. The mini-batch middle wins the trade.

The practical middle is B between 32 and a few thousand, chosen largely by what fills the hardware:

Name	Batch size	Gradient	Cost per step
full-batch gradient descent	$B = N$	exact	N per-example gradients
mini-batch SGD (today’s “SGD”)	32 to a few thousand	noisy estimate	B
strict SGD (the original sense)	$B = 1$	noisiest estimate	1

A naming caution. In modern papers and code, “SGD” means this mini-batch algorithm; the strict $B = 1$ reading of the name is historical.

5.6 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$\ell_n(\theta)$	example n 's loss, viewed as a function of the parameters
$\mathcal{B}$	the mini-batch itself: the set of examples drawn for one step
B	the batch size: how many examples $\mathcal{B}$ contains — a count, never the set
$\nabla L_{\mathcal{B}}$	the batch gradient: the average of $\nabla \ell_n$ over $\mathcal{B}$
epoch	one complete pass through the training data (N/B steps)
noise floor	the loss level just above the minimum that the bouncing cannot descend below

6 Overfitting and underfitting

Everything so far pushed one number down: the training loss. This section shows why that number is not the goal. It fits three models to one small dataset and lets new data judge them — exposing the two ways a trained model fails: *overfitting* (the model learns the training data’s accidents) and *underfitting* (the model misses the pattern). Every number is computed.

6.1 The goal is new data

A model is trained on N examples but used on inputs it has never seen — tomorrow’s photograph, the next sentence. Performance on new data is called *generalization*; it gets its own treatment in §9, and for now the plain meaning suffices: how well the trained model predicts examples that were not in its training set.

The training loss is only a stand-in for that goal, used because the training examples are the only data available. §3 already warned how far the stand-in can be pushed: reaching its global minimum usually means memorizing the training set. This section makes the warning concrete.

6.2 One dataset, three models

Take $N = 5$ measurements of the familiar rule $y = 2x$, with measurement noise (the normal condition of data, §5):

x_n	0	1	2	3	4
y_n	0.4	1.4	4.4	5.4	8.4
$r_n = y_n - 2x_n$	+0.4	−0.6	+0.4	−0.6	+0.4

The last row, the *residual* r_n , is each measurement’s miss from the rule — the noise. It is visible here only because this exercise knows the rule; a real dataset carries no such row.

Now fit three model families of increasing flexibility, each trained to its own minimum MSE (by gradient descent, or — for families this small — by direct solving; the result is the same):

- **A constant**, $\hat{y} = c$ (1 parameter). The best constant is the mean of the targets: $c = \frac{1}{5}(0.4 + 1.4 + 4.4 + 5.4 + 8.4) = 4.0$.
- **A line**, $\hat{y} = a + bx$ (2 parameters). Minimizing the MSE gives $a = 0$, $b = 2$: exactly $\hat{y} = 2x$, the rule itself.
- **A quartic** (a degree-4 polynomial), $\hat{y} = c_0 + c_1x + c_2x^2 + c_3x^3 + c_4x^4$ (5 parameters). Five parameters against five points can satisfy all five equations $\hat{y}(x_n) = y_n$ at once; solving them gives

$$\hat{y} = 0.4 - \frac{10}{3}x + \frac{20}{3}x^2 - \frac{8}{3}x^3 + \frac{1}{3}x^4,$$

a curve through every training point: training loss exactly 0.

Now compute the training loss of each of the three fitted models, starting each time from the MSE formula of §2, $L = \frac{1}{N} \sum_n (\hat{y}_n - y_n)^2$, here with $N = 5$.

The constant. Every prediction is $\hat{y}_n = 4$:

$$\begin{aligned}
L_{\text{const}} &= \frac{1}{5} \sum_{n=1}^5 (4 - y_n)^2 && \text{(insert } \hat{y}_n = 4) \\
&= \frac{1}{5} [(4-0.4)^2 + (4-1.4)^2 + (4-4.4)^2 + (4-5.4)^2 + (4-8.4)^2] && \text{(insert the targets)} \\
&= \frac{1}{5} [12.96 + 6.76 + 0.16 + 1.96 + 19.36] = \frac{41.2}{5} = 8.24. && \text{(square, add, average)}
\end{aligned}$$

The line. Every prediction is $\hat{y}_n = 2x_n$, so each gap $2x_n - y_n$ is the data table's residual row with the opposite sign:

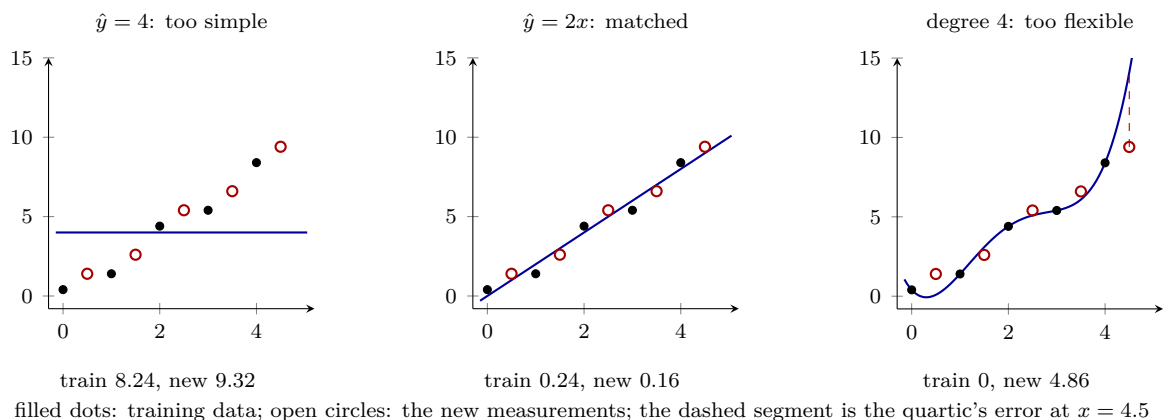
$$\begin{aligned}
L_{\text{line}} &= \frac{1}{5} \sum_{n=1}^5 (2x_n - y_n)^2 = \frac{1}{5} [(-0.4)^2 + 0.6^2 + (-0.4)^2 + 0.6^2 + (-0.4)^2] && \text{(the gaps are } -r_n) \\
&= \frac{1}{5} [0.16 + 0.36 + 0.16 + 0.36 + 0.16] = \frac{1.2}{5} = 0.24. && \text{(square, add, average)}
\end{aligned}$$

The quartic. It passes through every training point, so every prediction equals its target, $\hat{y}_n = y_n$, and every gap is zero: $L_{\text{quartic}} = \frac{1}{5} [0 + 0 + 0 + 0 + 0] = 0$.

Judged by training loss alone, the quartic is the best model ever fitted and the line is mediocre.

6.3 The verdict of new data

The next day brings five more measurements of the same rule, with new noise: (0.5, 1.4), (1.5, 2.6), (2.5, 5.4), (3.5, 6.6), (4.5, 9.4). The three models stay exactly as trained — nothing is refitted — and are evaluated on the new points:



Model	Parameters	Training MSE	New-data MSE
constant $\hat{y} = 4$	1	8.24	9.32
line $\hat{y} = 2x$	2	0.24	0.16
quartic (through all five points)	5	0	4.86

(One new-data loss populated in full, the line's: its predictions at the new x 's are 1, 3, 5, 7, 9, each missing the measurement by 0.4, so the loss is $\frac{1}{5} \cdot 5 \cdot 0.4^2 = 0.16$.) The table defines the two failures:

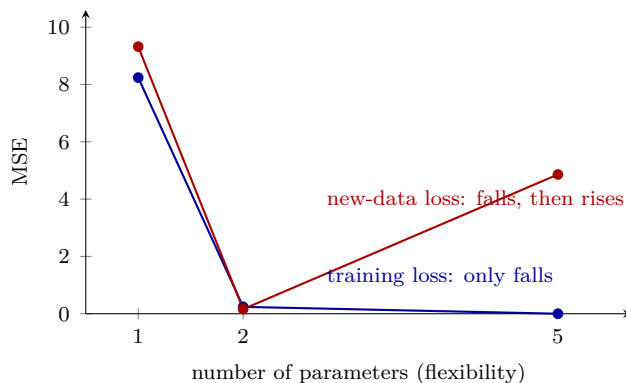
- A model *overfits* when it fits the training data more closely than the pattern supports, so its training loss is small but its new-data loss is far larger. The quartic: 0 against 4.86 — thirty times worse than the line it beat on the training set. The *gap* between the two losses is overfitting's signature.

- A model *underfits* when it is too simple to capture the pattern at all, so both losses are high. The constant: 8.24 against 9.32. Its signature: bad everywhere, no gap.

The line, whose form matches the rule, shows the healthy reading: both losses small and close (0.24 and 0.16 — that the new one is slightly *lower* is chance in where the noise landed).

What the quartic actually learned. Its extra parameters were spent explaining the residuals $+0.4, -0.6, \dots$ — the noise — as if they were pattern. That invented pattern is false the moment it leaves the training points: at $x = 4.5$ the quartic predicts 14.09 where the new measurement is 9.4, and that single error contributes $(14.09 - 9.4)^2 \approx 22.0$ — nine tenths of its entire new-data loss. The training set is stored perfectly; the rule is lost. This is the memorization §3 promised: the global minimum of the training loss — 0, actually reached — is worthless.

The two-loss picture, drawn against flexibility:



the three models of this section: underfit left, matched middle, overfit right

More flexibility *always* lowers the training loss — a larger family contains the smaller one, so its minimum can only be lower. The new-data loss instead falls while flexibility captures pattern, then rises when the surplus flexibility starts capturing noise. The best model sits at the turn, not at the right edge.

More data resists overfitting. The quartic could thread all five points because it had one parameter per point. Give the same quartic 50 points from the same rule and it can no longer thread them — its best fit is forced back toward the rule. Overfitting is not a property of a model alone but of *capacity relative to data*.

6.4 Capacity, and telling the failures apart

Capacity is a model's flexibility: how wide a family of functions it can realize. Parameter count is a rough measure — the constant, line, and quartic above are capacities 1, 2, 5. Too little capacity for the pattern underfits; too much for the available data can overfit; matched, the model generalizes. Modern neural networks sit at enormous capacity, which is why overfitting is a permanent concern in training them.

For a real model the pictures above cannot be drawn, so the failures are told apart exactly as the table told them apart — by comparing two losses:

- training loss low, new-data loss high: overfitting;
- both high: underfitting;

- both low and close: healthy.

Measuring a “new-data loss” during development requires setting data aside from training so that it can play the role of new data — the train/validation/test splits of §8.

Two earlier promises are now redeemable. §3: the global minimum of the training loss is not the goal — the quartic reached it and lost. §5: SGD’s noise often *helps* generalization — the bouncing walk resists settling into the sharpest, most memorization-like minima, a (still partly empirical) reason noisy training tends to generalize better than exact descent.

6.5 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
generalization	how well the trained model predicts examples not in its training set
overfitting	fitting the training data more closely than the pattern supports
underfitting	being too simple to capture the pattern at all
r_n	the residual: measurement n ’s miss from the rule, $y_n - 2x_n$
capacity	a model’s flexibility; parameter count is a rough measure

7 The bias–variance trade-off

§6 diagnosed two failures by comparing losses. This section explains the same two failures by asking a sharper question: *what would the fitted model have been, had the training data come out differently?* Answering it splits a model family’s error into two named parts — *bias*, the error it always makes, and *variance*, the error that changes with the draw — and shows that underfitting is too much bias and overfitting is too much variance.

7.1 Same family, different draw

Training data is itself a sample: measure the same rule tomorrow and the noise lands differently. Make the noise explicit this time: each measurement of the rule $y = 2x$ carries noise ε that is $+0.4$ with probability 0.6 and -0.6 with probability 0.4 (so its mean is $0.6 \cdot 0.4 - 0.4 \cdot 0.6 = 0$). §6’s training set was one draw from this rule — call it *draw A*. Here is a second draw at the same inputs, with fresh noise (*draw B*):

x_n	0	1	2	3	4
y_n (draw A)	0.4	1.4	4.4	5.4	8.4
y_n (draw B)	-0.6	2.4	3.4	6.4	7.4

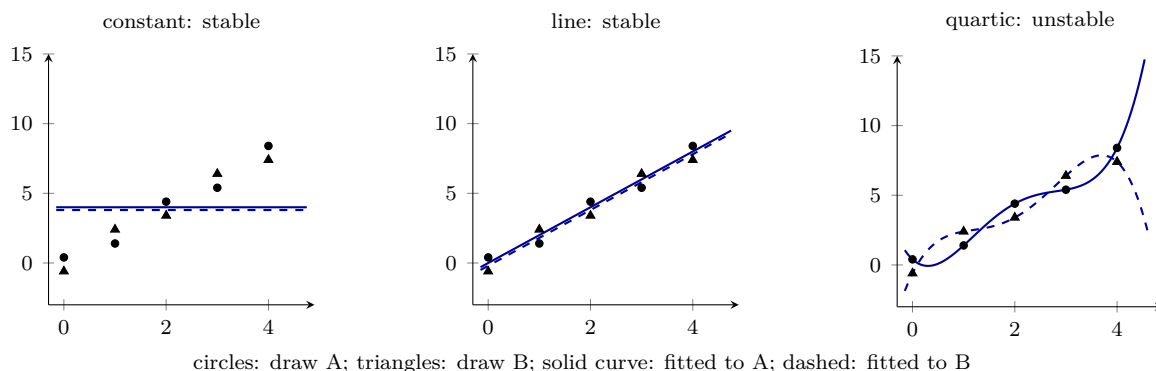
Fit the same three families to draw B, exactly as §6 fitted them to draw A: the best constant is the mean, $\frac{1}{5}(-0.6 + 2.4 + 3.4 + 6.4 + 7.4) = 3.8$; minimizing the MSE for the line gives $\hat{y} = 2x - 0.2$; and solving the five pass-through-every-point equations for the quartic gives

$$\hat{y} = -\frac{3}{5} + \frac{22}{3}x - \frac{20}{3}x^2 + \frac{8}{3}x^3 - \frac{1}{3}x^4.$$

Now place each family’s two fits side by side:

Family	Trained on draw A		Trained on draw B	
	fitted model	$\hat{y}(4.5)$	fitted model	$\hat{y}(4.5)$
constant	$\hat{y} = 4.0$	4.0	$\hat{y} = 3.8$	3.8
line	$\hat{y} = 2x$	9.0	$\hat{y} = 2x - 0.2$	8.8
quartic	through A’s points	14.09	through B’s points	3.71

(For reference, the rule’s value at $x = 4.5$ is 9.) The constant and the line barely notice which draw they were trained on. The quartic is a different curve entirely each time: at $x = 4.5$ its two versions disagree by more than 10 — one soars, the other dives:



7.2 Bias and variance

Imagine continuing this experiment over many draws and *averaging* each family's fitted models. Two draws already show where the averages head. Averaging the two constants gives the flat line 3.9. Averaging the two lines gives $2x - 0.1$. Averaging the two quartics, coefficient by coefficient, gives

$$\frac{1}{2}\left(\frac{2}{5} - \frac{3}{5}\right) = -\frac{1}{10}, \quad \frac{1}{2}\left(-\frac{10}{3} + \frac{22}{3}\right) = 2, \quad \frac{1}{2}\left(\frac{20}{3} - \frac{20}{3}\right) = 0, \quad \frac{1}{2}\left(-\frac{8}{3} + \frac{8}{3}\right) = 0, \quad \frac{1}{2}\left(\frac{1}{3} - \frac{1}{3}\right) = 0,$$

so the averaged model, assembled from these coefficients, is

$$\hat{y} = -\frac{1}{10} + 2x + 0 \cdot x^2 + 0 \cdot x^3 + 0 \cdot x^4 = 2x - 0.1$$

— the wiggles cancel, and the average quartic is the *line* $2x - 0.1$, the same average the line family produced. The two names attach to what this experiment separates:

- The *bias* of a model family compares its *average* fit with the *truth*: how far the fit stays from the rule even after the draw-to-draw differences are averaged away. (E.g. the constants: their average, the flat 3.9, still misses the rule's 8 at $x = 4$ by 4.1 — averaging removed nothing, because every draw is flat.)
- The *variance* of a model family is the *spread* of its fits around that average: how much the fitted model changes from draw to draw. It does not measure distance to the truth at all — it measures the distance of the fits *from each other*; the rule does not appear in it. (E.g. the quartics at $x = 4.5$: 14.09 and 3.71, a spread of more than 10 around their average 8.9 — even though that average itself is nearly right, the rule saying 9.)

The three families, judged:

- **Constant: high bias, low variance.** Its average fit is flat near 4 — and flat can never follow a rule that climbs. No amount of extra data or averaging repairs this; the error is built into the family. Each single draw is far from the rule too, but far in the *same way* every time ($4.0 \rightarrow 3.8$): the fits agree with each other, so the whole error is booked as bias.
- **Line: low bias, low variance.** Its average fit is essentially the rule, and single draws sit close to that average ($2x$ versus $2x - 0.2$).
- **Quartic: low bias, high variance.** Its average fit is *also* essentially the rule — the quartic's problem was never a systematic error. Its problem is the spread: single draws swing more than 10 apart at $x = 4.5$. The overfitting of §6 is this spread, seen in one draw.

7.3 The decomposition

Bias and variance are not only descriptive words: the error can be *computed* and split into these parts. This subsection does it twice — first with the concrete numbers of draw A, then in general. Recall the residual row of §6.2: it defined $r_n = y_n - 2x_n$. Adding $2x_n$ to both sides turns it into $y_n = 2x_n + r_n$: each measurement is the rule's value plus its miss. Insert this into the constant's gap, prediction minus target:

$$4 - y_n = 4 - (2x_n + r_n) = (4 - 2x_n) - r_n,$$

and the loss separates with it:

$$\begin{aligned}
L_{\text{const}} &= \frac{1}{N} \sum_{n=1}^N (\hat{y}_n - y_n)^2 = \frac{1}{5} \sum_{n=1}^5 (4 - y_n)^2 && \text{(the MSE; insert } \hat{y}_n = 4, N = 5) \\
&= \frac{1}{5} \sum_{n=1}^5 [(4 - 2x_n) - r_n]^2 && \text{(split each gap: } y_n = 2x_n + r_n) \\
&= \frac{1}{5} \sum_n (4 - 2x_n)^2 + \frac{1}{5} \sum_n r_n^2 - \frac{2}{5} \sum_n (4 - 2x_n) r_n && \text{(expand the square)}
\end{aligned}$$

and, inserting the five values of each sum,

$$\begin{aligned}
L_{\text{const}} &= \frac{1}{5} [16 + 4 + 0 + 4 + 16] + \frac{1}{5} [0.16 + 0.36 + 0.16 + 0.36 + 0.16] \\
&\quad - \frac{2}{5} [1.6 - 1.2 + 0 + 1.2 - 1.6] \\
&= 8 + 0.24 - 0 = 8.24
\end{aligned}$$

Read the result: the 8.24 that §6 measured for the constant is really two numbers added together. The 8 comes from the first sum: the distance between the flat prediction 4 and the rule's value $2x_n$, squared and averaged. This is the systematic part — it would appear again, unchanged, in every draw. The 0.24 comes from the second sum: the residuals themselves, squared and averaged. This is the noise part. (The third sum is exactly 0 on this draw because these residuals happen to satisfy $\sum_n r_n = 0$ and $\sum_n x_n r_n = 0$; on another draw it would only be small, and it is zero on average.) The same split explains the line: its prediction $2x_n$ has distance 0 from the rule at every input, so its systematic part is 0, and its whole 0.24 is noise.

The general statement uses expectation and variance (the tools of *Essential Math for AI*). A new measurement at input x is $y = 2x + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$ and $\text{Var}(\varepsilon) = \sigma^2$. Because the mean is zero, the variance is also the mean square — by the identity $\mathbb{E}[X^2] = \mathbb{E}[X]^2 + \text{Var}(X)$,

$$\mathbb{E}[\varepsilon^2] = \mathbb{E}[\varepsilon]^2 + \text{Var}(\varepsilon) = 0 + \sigma^2 = \sigma^2,$$

the form the derivation below uses. Two more facts set the stage. The prediction $\hat{y}$ is also random, because it depends on which training set was drawn. And the new measurement's noise ε is independent of that draw — the training happened before this measurement was made. The symbol $\mathbb{E}$ below averages over both sources of randomness. Then:

$$\begin{aligned}
\mathbb{E}[(\hat{y} - y)^2] &= \mathbb{E}[(\hat{y} - 2x - \varepsilon)^2] && \text{(insert } y = 2x + \varepsilon) \\
&= \mathbb{E}[(\hat{y} - 2x)^2 - 2(\hat{y} - 2x)\varepsilon + \varepsilon^2] && \text{(expand the square)} \\
&= \mathbb{E}[(\hat{y} - 2x)^2] - 2\mathbb{E}[(\hat{y} - 2x)\varepsilon] + \mathbb{E}[\varepsilon^2] && \text{(\mathbb{E} of a sum = sum of \mathbb{E}'s)} \\
&= \mathbb{E}[(\hat{y} - 2x)^2] - 2\mathbb{E}[\hat{y} - 2x]\mathbb{E}[\varepsilon] + \mathbb{E}[\varepsilon^2] && \text{(independence: } \mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]) \\
&= \mathbb{E}[(\hat{y} - 2x)^2] + \sigma^2 && (\mathbb{E}[\varepsilon] = 0, \mathbb{E}[\varepsilon^2] = \sigma^2) \\
&= (\mathbb{E}[\hat{y} - 2x])^2 + \text{Var}(\hat{y} - 2x) + \sigma^2 && (\mathbb{E}[X^2] = \mathbb{E}[X]^2 + \text{Var}(X)) \\
&= (\mathbb{E}[\hat{y}] - 2x)^2 + \text{Var}(\hat{y}) + \sigma^2 && (2x \text{ constant: mean shifts, spread stays})
\end{aligned}$$

that is,

$$\mathbb{E}[(\hat{y} - y)^2] = \underbrace{(\mathbb{E}[\hat{y}] - 2x)^2}_{\text{bias}^2} + \underbrace{\text{Var}(\hat{y})}_{\text{variance}} + \underbrace{\sigma^2}_{\text{irreducible}}.$$

Each of the three terms has a clear owner. The first term is the squared bias: how far the family’s average fit is from the rule. For the constant family this is the 8 computed above. The second term is the variance: how much the fits spread around their average. This is what makes the quartic fail. The third term, σ^2 , belongs to no model: it is the noise in the *new* measurement itself, and no model can predict noise that has not happened yet. Its value follows from the noise model of §7.1 — ε is +0.4 with probability 0.6 and −0.6 with probability 0.4:

$$\begin{aligned}
 \sigma^2 &= \text{Var}(\varepsilon) = \mathbb{E}[\varepsilon^2] - \mathbb{E}[\varepsilon]^2 && \text{(the definition of variance)} \\
 &= \mathbb{E}[\varepsilon^2] - 0^2 = \mathbb{E}[\varepsilon^2] && \text{(the mean is 0)} \\
 &= \sum_{\text{values } v} P(\varepsilon = v) \cdot v^2 && \text{(each value squared, weighted)} \\
 &= \underbrace{0.6}_{P(\varepsilon=+0.4)} \cdot (+0.4)^2 + \underbrace{0.4}_{P(\varepsilon=-0.6)} \cdot (-0.6)^2 && \text{(insert the two values)} \\
 &= 0.6 \cdot 0.16 + 0.4 \cdot 0.36 = 0.096 + 0.144 = 0.24. && \text{(square, multiply, add)}
 \end{aligned}$$

Even a model that predicts $2x$ exactly — the rule itself — still makes an average squared error of 0.24 on new measurements. The line’s two losses (0.24 on the training data, 0.16 on the new data) are already at this level: the line had learned everything that could be learned.

7.4 The trade-off

The decomposition explains the two-loss picture of §6. When a model family becomes more flexible, two things happen at the same time: the bias falls, because a wider family can follow more of the rule; and the variance rises, because a wider family can also follow more of the noise. The new-data loss is the sum of both plus the fixed σ^2 . So, as flexibility grows, the new-data loss first falls — the falling bias dominates — and then rises — the rising variance takes over. The best family sits at the balance point between the two; in §6’s example, that was the line. Choosing a model’s capacity means choosing a place on this curve. In the overview’s words: underfitting is too much bias, overfitting is too much variance.

One caution about modern practice. Very large neural networks — networks with far more parameters than training examples — often generalize better than this classic curve predicts, and why they do is still an open research question. The vocabulary of this section remains the standard way to name the two error sources; the curve itself should not be applied blindly to the largest models.

7.5 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
draw	one possible training set, sampled from the same rule with fresh noise
bias	the error of the family’s average fit: systematic, survives averaging
variance	the spread of the fits around their average, across draws
ε , σ^2	the noise in one measurement, and its variance (0.24 here)
irreducible error	σ^2 : the part of the new-data error no model can remove

8 Train, validation, and test splits

§6 judged models with new data that conveniently arrived the next day. A real project cannot wait for new data. Instead it creates new data out of the data it already has, by holding part of it back. This section explains the standard division of a dataset into three parts — training, validation, and test — and computes exactly why the third part is necessary.

8.1 Creating new data by holding some back

The idea is simple. Before training starts, set some examples aside and never show them to the update rule. The model’s parameters are then fitted without those examples, so when the fitted model is finally scored on them, they are new data to it — data it has never influenced. Examples kept aside this way are called *held out*.

§6 already followed this pattern without naming it: five points trained the three models, and five separate points judged them. The only difference in practice is that both parts come from one dataset: shuffle the examples (as in §5), cut the shuffled list in two, train on the larger part, and judge on the smaller.

8.2 Why two parts are not enough

If training the parameters were the whole job, two parts would suffice. But the chapter has been collecting settings that the update rule does *not* learn — the hyperparameters: the learning rate η (§4), the batch size B (§5), the model’s capacity (§6). The person chooses these, and the natural way to choose is to try several candidates and keep the one that scores best on held-out data.

That choosing step creates a problem. A score on a small set is partly luck — and by keeping the best-scoring candidate, the person systematically keeps lucky scores. After the choice, the winner’s score on that set is too good to be trusted; the next subsection computes how much too good. Data that has influenced a choice is no longer new. The standard answer is to hold out *two* parts with different jobs:

- The *training set* is the part of the data that the parameters are fitted on.
- The *validation set* is a held-out part used to compare candidate settings and choose among them.
- The *test set* is a held-out part that is used exactly once, at the very end, to report the final model’s quality.

Part	Typical share	Used by	Touched	Decides
training	80%	the update rule	every step	the parameters θ
validation	10%	the person comparing	many times	hyperparameters; when to stop
test	10%	nobody until the end	once	the reported quality

The overview’s phrasing can now be read literally: train on one part, tune on the second, report on a third that the model — including every choice made about it — never influenced.

8.3 The leak, computed

Why exactly does choosing spoil a score? The chapter’s own numbers show the ingredient: luck. §7 computed that the line’s true expected error is the noise floor $\sigma^2 = 0.24$ — yet in §6 the line

scored 0.16 on the five next-day points. Both numbers are correct. A score measured on a *small* set is the true error plus the luck of how the noise fell on those few points; there it fell favorably, by -0.08 .

Turn this into the smallest possible model of scoring (two outcomes keep the computation small, the same simplification as §7’s noise model): a candidate whose true error is 0.24, measured on a small validation set, scores

$$0.24 - 0.08 = 0.16 \quad (\text{lucky}) \quad \text{or} \quad 0.24 + 0.08 = 0.32 \quad (\text{unlucky}),$$

each with probability $\frac{1}{2}$.

Measuring one candidate is honest. The expected score is the expectation formula of *Essential Math for AI* — each value weighted by its probability:

$$\begin{aligned} \mathbb{E}[\text{score}] &= \sum_{\text{values } v} P(\text{score} = v) \cdot v && (\text{the expectation formula}) \\ &= \frac{1}{2} \cdot 0.16 + \frac{1}{2} \cdot 0.32 = 0.08 + 0.16 = 0.24. && (\text{insert the two scores}) \end{aligned}$$

On average the measurement returns the true error. Luck is present but does not favor a direction.

Choosing between two candidates is not honest. Take two candidate settings that are *equally good* — both truly err 0.24 — score both on the same validation set, and keep the better score. (Equally good candidates exist: §4’s $\eta = 0.1$ and $\eta = 0.3$ produced identical loss curves.) Each candidate’s luck is independent, so four outcomes are equally likely, and the kept score is the smaller of the two:

	candidate 1 scores	candidate 2 scores	kept (smaller)	probability
both lucky	0.16	0.16	0.16	1/4
only 1 lucky	0.16	0.32	0.16	1/4
only 2 lucky	0.32	0.16	0.16	1/4
both unlucky	0.32	0.32	0.32	1/4

The expected kept score, by the same expectation formula:

$$\begin{aligned} \mathbb{E}[\text{kept score}] &= \sum_{\text{values } v} P(\text{kept} = v) \cdot v && (\text{the expectation formula}) \\ &= \frac{3}{4} \cdot 0.16 + \frac{1}{4} \cdot 0.32 = 0.12 + 0.08 = 0.20. && (\text{kept} = 0.16 \text{ in three of four rows}) \end{aligned}$$

Both candidates truly err 0.24, but the reported winner averages 0.20. Choosing made the number optimistic by 0.04 — although nothing about the *models* improved.

More candidates, more optimism. With ten equally good candidates, the kept score is 0.32 only if all ten are unlucky at once:

$$\begin{aligned} \mathbb{E}[\text{kept score}] &= \left(1 - \frac{1}{2^{10}}\right) \cdot 0.16 + \frac{1}{2^{10}} \cdot 0.32 && (\text{all ten unlucky: probability } 1/2^{10} = 1/1024) \\ &= \frac{1023}{1024} \cdot 0.16 + \frac{1}{1024} \cdot 0.32 \approx 0.1602. \end{aligned}$$

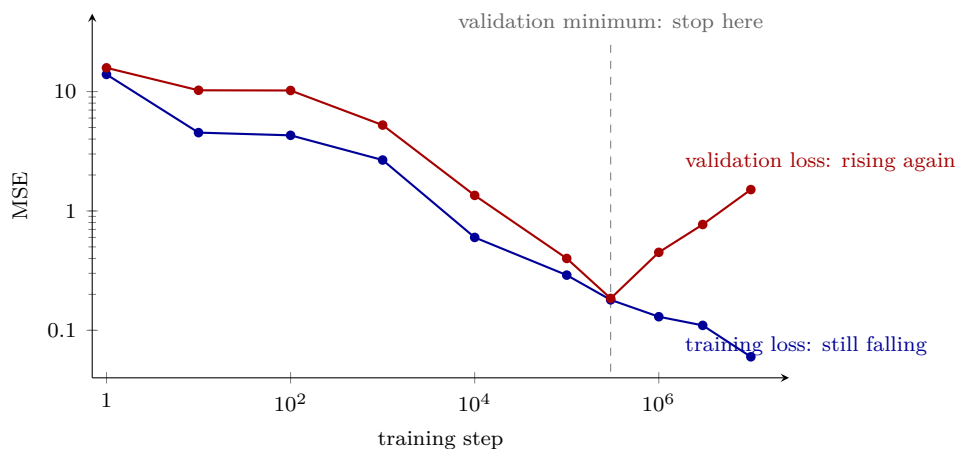
The report says 0.16 for a model that truly errs 0.24. This gain of optimism without any gain of quality is called a *leak*: the validation set’s luck has flowed into the choice, and its score stopped being a measurement.

The test set repairs the damage. The chosen winner still truly errs 0.24, and on a set that took part in *no* choice, the honest computation above applies again: expected score 0.24. This is also why the test set is used only once — the moment a second model is compared against it, the table above starts over with the test set in the validation set’s role.

8.4 Splits in practice

How to split. Split once, at random, before anything is trained: shuffle the examples, then cut. Classic shares are 80/10/10. For very large datasets the held-out shares can be much smaller, because what makes a score stable is the absolute number of held-out examples, not their percentage — 10,000 examples give a steady score whether they are 10% of the data or 0.1%.

The validation set works during training too. Its second job is to watch training as it runs: after every stretch of steps, score the current model on the validation set. This produces the most important diagnostic plot in practice — the training loss and the validation loss, side by side, against training time. Here it is for a run the chapter can build itself: the quartic of §6, fitted to draw A by gradient descent (all five coefficients starting at 0, $\eta = 10^{-5}$), scored along the way on the next-day points. The curves are computed by actually running this training; the recipe above reproduces them:



both axes are logarithmic: each horizontal tick is 100 times more steps, each vertical tick 10 times more loss

Read the two curves with §6’s vocabulary. The training loss falls the whole way — gradient descent guarantees nothing else. The validation loss falls with it, reaches its minimum at step $3 \cdot 10^5$ (validation 0.185, training 0.179 — at this moment the quartic behaves almost like the line), and then rises: from here on, training is fitting draw A’s noise. By step 10^7 the training loss has improved to 0.06 while the validation loss has worsened to 1.51, on its way toward the interpolant’s 4.86. Training longer improved the training loss and damaged the model. Ending training at the validation minimum instead of the training minimum is called *early stopping*, and it is the simplest cure for overfitting in daily use.

Two closing rules. For very small datasets there is a refinement called *cross-validation* — rotate which part is held out and average the scores — that a statistics text covers in full; large-scale practice mostly uses the plain three-way split. And the test set is spent by use: score on it once, report, and do not touch it again.

8.5 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
held out	kept away from training, so it can play the role of new data
training set	the part of the data the parameters are fitted on
validation set	a held-out part used to compare candidate settings and choose
test set	a held-out part used exactly once, at the end, for the report
leak	luck flowing from a dataset into a choice, making its score optimistic
early stopping	ending training at the validation minimum, not the training minimum

9 Generalization

Every tool of this chapter — the loss, the walk downhill, the held-out splits — serves one goal that has so far only been glossed: predicting well on data the model has never seen. This section gives that goal its proper name and its measure, states the assumption that makes it achievable at all, and shows what happens when the assumption breaks.

9.1 The measure

Generalization is a model’s ability to predict well on data it was not trained on. It is measured with the two losses the chapter already tracks, and their difference has its own name — the *generalization gap*:

$$\text{gap} = L_{\text{new}} - L_{\text{train}},$$

the new-data loss minus the training loss. For §6’s line, populated: $\text{gap} = 0.16 - 0.24 = -0.08$. All three models:

Model	L_{train}	L_{new}	gap	reading
constant	8.24	9.32	+1.08	bad, and honestly reported
line	0.24	0.16	−0.08	good, and honestly reported
quartic	0	4.86	+4.86	the training loss lied

Read the two numbers separately, because they answer different questions. The *new-data loss* is the goal itself: only the line has it low. The *gap* says whether the training loss can be believed: the constant’s small gap means its training loss told the truth — the model is bad, and training already showed it — while the quartic’s huge gap is §6’s overfitting signature. When practitioners say a model “generalizes well”, they mean the new-data loss is low; the gap is the diagnostic tool along the way.

9.2 The assumption that makes it possible

Why should skill on one set of examples transfer to another set at all? Because both sets come from the *same source*. The chapter’s source has been explicit since §7: draw an input x , apply the rule $2x$, add noise ε (+0.4 with probability 0.6, −0.6 with probability 0.4). A source of examples — which inputs occur, and which targets they arrive with — is called a *distribution*. Examples produced by drawing from one distribution, each draw not influencing the others, are called *independent and identically distributed* — abbreviated *i.i.d.*, the standard shorthand.

This assumption is what the entire measuring machinery of §8 stands on. A held-out score estimates new-data performance *only if* the held-out examples come from the same distribution as the training examples — which the standard procedure guarantees by construction: one dataset, shuffled, then cut, so every part is a draw from the same source. A model that captured the rule does well on every fresh draw from that source. That much — and only that much — is what generalization promises.

9.3 When the assumption breaks: distribution shift

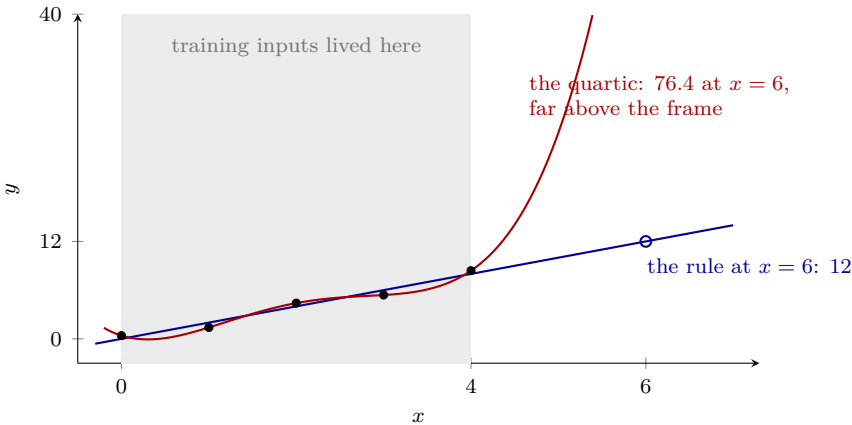
Distribution shift is the situation where the data met in use comes from a different distribution than the data trained on. It is common in practice: a vision model trained on daytime photographs is deployed at night; a spam filter trained on last year’s messages reads this year’s; a shop’s demand model trained before a price change predicts after it. Under shift, the test score loses its meaning

— the test set was drawn from the *old* distribution, so it certifies performance on a world that is no longer the one the model faces.

The chapter contains a small shift of its own: the input range. The training inputs lived between 0 and 4. Ask the two surviving models about $x = 6$, an input from outside that region. The line answers with the rule’s own value, $\hat{y} = 2 \cdot 6 = 12$. The quartic answers with

$$\hat{y} = 0.4 - \frac{10}{3} \cdot 6 + \frac{20}{3} \cdot 36 - \frac{8}{3} \cdot 216 + \frac{1}{3} \cdot 1296 = 0.4 + \frac{-60+720-1728+1296}{3} = 0.4 + \frac{228}{3} = 76.4,$$

more than six times the rule’s value:



inside the shaded region both models look usable; outside it, only the model that matched the rule survives

Honesty requires the other half of this statement. The line survives at $x = 6$ only because the rule $y = 2x$ actually continues out there — and the data never said anything about inputs beyond 4. If the true relation bent after $x = 4$, the line would fail too, and no score computed inside the training region would give a warning. Predictions outside the training distribution are *extrapolation*: they rest on the assumption that the world’s rule extends, not on anything the data verified.

9.4 The levers

Everything this chapter built moves this one number — the new-data loss. Gathered in one place:

Lever	What it does	Where
more training data	makes memorizing impossible; shrinks variance	§6, §7
capacity matched to data	balances bias against variance	§6, §7
early stopping	ends training at the validation minimum	§8
SGD’s noise	resists the memorization-like minima	§5

One lever remains: penalizing complexity directly, inside the loss itself, while training runs. It is called *regularization*, and it is §10.

9.5 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
generalization	a model's ability to predict well on data it was not trained on
generalization gap	$L_{\text{new}} - L_{\text{train}}$: how much the training loss understated
distribution	a source of examples: which inputs occur, with which targets
i.i.d.	drawn from one distribution, no draw influencing another
distribution shift	the data met in use comes from a different source than training
extrapolation	predicting outside the region the training inputs covered

10 Regularization

§9 ended with one lever still unexplained: penalizing complexity inside the loss itself. This section builds that lever. It first shows, on the chapter’s own models, that complexity is visible in the *size of the parameters*; then it adds a size penalty to the loss (the L2 and L1 penalties), derives what the penalty does to the update rule, and demonstrates on the quartic that one continuous dial — the penalty’s strength — replays the whole underfit-to-overfit story of §6.

10.1 Complexity is visible in the parameters

Place §6’s two fitted models side by side, coefficient by coefficient:

Model	c_0	c_1	c_2	c_3	c_4	$\sum_j c_j^2$
line	0	2	0	0	0	4
quartic	0.4	−3.33	6.67	−2.67	0.33	62.94

The wiggly model is built from *large, opposing* coefficients: −3.33, +6.67, −2.67 pull against each other, and their tug is exactly what bends the curve up and down between the training points. The smooth model needs only small numbers. This is general: bending costs coefficient size, smoothness is cheap. So a preference for simple functions can be written as a preference for *small parameters* — and a preference can be added to a loss.

Regularization is a change to the training loss that adds a penalty for large parameters, so that training favors simple functions as well as accurate ones:

$$L_{\text{reg}}(\theta) = L(\theta) + \lambda \cdot \text{penalty}(\theta).$$

The number λ (“lambda”) is the *regularization strength*: it sets the price of parameter size relative to the price of wrongness. It is a hyperparameter in §4’s sense, chosen on the validation set (§8). Two penalties are standard, named after the two standard ways of measuring a vector’s length.

10.2 L2: penalize the squares

The *L2 penalty* charges the sum of squared parameters — the squared length of the parameter vector:

$$L_{\text{reg}}(\theta) = L(\theta) + \lambda \sum_j \theta_j^2.$$

What does this do to training? Differentiate the penalty with respect to one parameter θ_j (only the $k = j$ term of the sum depends on it; power rule):

$$\frac{\partial}{\partial \theta_j} \left[\lambda \sum_k \theta_k^2 \right] = \lambda \cdot 2 \theta_j = 2\lambda \theta_j,$$

and insert the combined gradient into the update rule of §3:

$$\begin{aligned} \theta_{\text{new}} &= \theta_{\text{old}} - \eta \nabla L_{\text{reg}}(\theta_{\text{old}}) && \text{(the update rule)} \\ &= \theta_{\text{old}} - \eta [\nabla L(\theta_{\text{old}}) + 2\lambda \theta_{\text{old}}] && \text{(the two gradients add)} \\ &= (1 - 2\eta\lambda) \theta_{\text{old}} - \eta \nabla L(\theta_{\text{old}}). && \text{(collect the } \theta_{\text{old}} \text{ terms)} \end{aligned}$$

Read the last line as a two-part step: first every parameter is multiplied by the factor $(1 - 2\eta\lambda)$ — shrunk slightly toward 0 — and then the usual gradient step is applied. Populated with $\eta = 0.1$ and

$\lambda = 0.05$: the factor is $1 - 2 \cdot 0.1 \cdot 0.05 = 0.99$, so every step begins by multiplying every parameter by 0.99. This is why L2 regularization is also called *weight decay*: the weights decay a little at every step, and only the parameters that the data actively defends stay large. (In deep-learning practice the name weight decay is the common one — it appears as a setting inside optimizers, with typical λ between 10^{-4} and 10^{-2} . A third name, *ridge regression*, means least squares with the L2 penalty.)

Note one property: a proportional shrink never reaches zero. Multiplying by 0.99 again and again makes a parameter tiny, but never exactly 0.

10.3 L1: penalize the absolute values

The *L1 penalty* charges the sum of absolute values:

$$L_{\text{reg}}(\theta) = L(\theta) + \lambda \sum_j |\theta_j|.$$

The derivative of $|\theta_j|$ is +1 when $\theta_j > 0$ and -1 when $\theta_j < 0$ (at exactly 0 it has a kink and no derivative; software treats it as 0). So the penalty’s pull is a *fixed amount* $\eta\lambda$ per step, always toward 0 — unlike L2’s pull, which is proportional to the parameter and fades as the parameter shrinks. The difference decides whether zero is ever reached. One parameter, starting at 1, under the penalty alone ($\eta\lambda = 0.01$ for L1, factor 0.99 for L2):

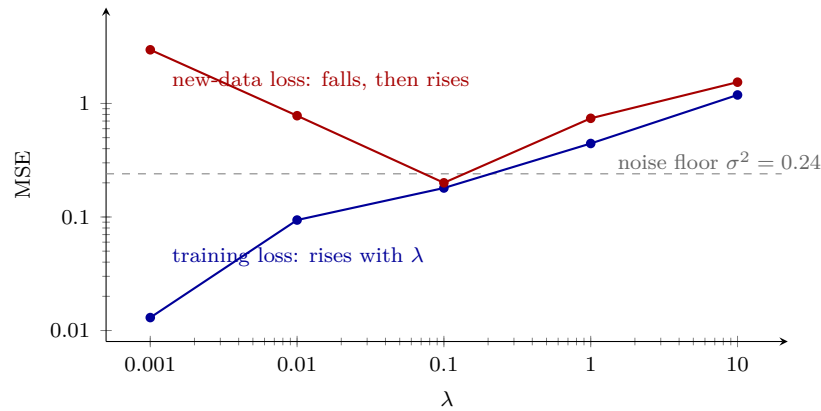
step	0	1	2	100	1000
L2 (multiply by 0.99)	1	0.99	0.9801	0.366	0.00004
L1 (subtract 0.01)	1	0.99	0.98	0	0

L1 walks its parameters to *exactly* zero and holds them there — a parameter stays nonzero only if the data’s gradient pushes back harder than λ . A parameter vector with many exact zeros is called *sparse*, and this is L1’s use: the trained model itself reports which inputs it did not need. L2 is the default choice for generalization; L1 is chosen when sparsity is wanted.

10.4 The dial, demonstrated

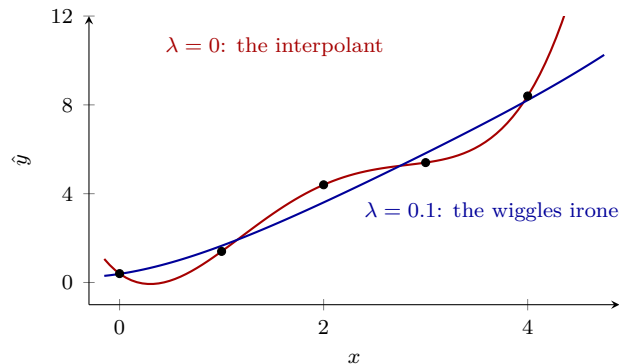
Now regularize the chapter’s own overfitter. Take the quartic family on draw A, train it by minimizing the L2-regularized loss (for a family this small the minimum can be solved directly; gradient descent reaches the same answers), and judge each result on the next-day points, exactly as §6 judged. Sweep λ from 0 upward:

λ	L_{train}	L_{new}	$\sum_j c_j^2$	verdict
0	0	4.86	62.9	§6’s interpolant: overfit
0.001	0.013	2.97	31.9	still overfit
0.01	0.094	0.78	4.29	most wiggle gone
0.1	0.180	0.20	1.16	the best of the sweep
1	0.444	0.74	0.31	shrunk too far
10	1.188	1.54	0.09	pushed toward 0: underfit



the quartic family under L2 regularization, judged on the next-day points; both axes logarithmic

Read the table’s best row against §6. The same quartic family that scored 0 train, 4.86 new when unregularized scores 0.18 train, 0.20 new at $\lambda = 0.1$ — the quality of the line, reached without changing the family at all. The fitted coefficients tell how: they shrank from $(0.4, -3.33, 6.67, -2.67, 0.33)$ to $(0.39, 0.70, 0.71, -0.15, 0.01)$, and the wiggles went with them:



In §7’s vocabulary: the penalty added a little bias (the training loss rose from 0 to 0.18) and removed a great deal of variance (the new-data loss fell from 4.86 to 0.20, close to the noise floor 0.24). And where §6 could only jump between three separate families — 1, 2, or 5 parameters — λ moves *continuously* from the interpolant ($\lambda = 0$) toward everything-near-zero (λ large): a capacity dial that needs no change of architecture. Its setting is chosen like every hyperparameter: compare candidates on the validation set, and remember §8 — the comparison is a choice, so the winner’s honest score comes from the test set.

One last widening of the word. L2 and L1 are the *explicit* regularizers — terms written into the loss. The chapter has already met *implicit* ones: early stopping (§8) and SGD’s noise (§5) also restrict how closely training can fit, without touching the loss. In practice “regularization” covers every such device, and large models are trained with several at once.

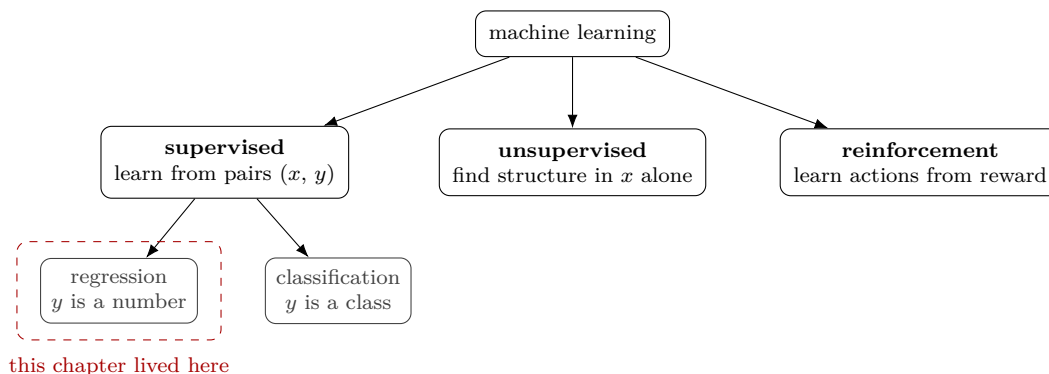
10.5 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
λ	the regularization strength: the price of parameter size
L_{reg}	the regularized loss: L plus λ times the penalty
L2 penalty	$\sum_j \theta_j^2$: proportional pull, never exactly zero (weight decay)
L1 penalty	$\sum_j \theta_j $: fixed pull, reaches exactly zero
sparse	a parameter vector with many exact zeros

11 The map of machine learning

Everything so far happened inside one corner of machine learning: examples came as input–target pairs, and the loss compared predictions with targets. This section names that corner and draws the whole map — the standard division of machine learning into *supervised*, *unsupervised*, and *reinforcement* learning — so that the chapter’s tools can be placed, and their reach beyond this chapter stated honestly.



11.1 Supervised learning

Supervised learning is learning from examples that come with answers: every training example is a pair (x, y) — an input and the correct target for it — and the loss measures how far the model’s prediction falls from the target. The target is also called the *label*, and a dataset of such pairs is called *labeled*. The word “supervised” is literal: a supervisor (usually a human who labeled the data) has said what the right answer is for every training input.

The two sub-tasks were both met in §2, split by what kind of thing y is:

Task	The target y is	Loss of §2	Example
regression	a number	MSE	price of a house, tomorrow’s temperature
classification	a class from a list	cross-entropy	cat/dog/bird, spam/not-spam

The whole chapter — the landscape, the walk, the splits, the dial — was built on supervised regression with the running pair (x, y) . That was not a limitation of the tools; it was the simplest place to build them.

11.2 Unsupervised learning

Unsupervised learning is learning from examples that come without answers: the dataset holds only inputs x , no targets, and the task is to find structure in the inputs themselves. Nobody says what the right answer is — there is no right answer per example — yet there is still a loss, built from the inputs alone. The typical tasks:

- *Clustering*: group similar examples together — for instance, divide a shop’s customers into a few segments that behave alike, without anyone defining the segments beforehand.
- *Dimensionality reduction*: compress each example into fewer numbers while keeping the variation that matters — a summary the model chooses itself.

- *Anomaly detection*: learn what typical examples look like, then flag the untypical — a strange credit-card transaction, a failing machine’s sensor readings.

One modern case deserves its own name. Training a language model to predict the next word of raw text needs no human labels — the “target” is simply the next word, cut out of the text itself. Formally each example is still a pair (x, y) , so the machinery is supervised; but since the labels are manufactured from the raw data instead of written by people, this is called *self-supervised learning*. It is the trick that lets the largest models train on datasets no human could ever label, and a later chapter on language models builds on it.

11.3 Reinforcement learning

Reinforcement learning is learning to act: an *agent* (the learner) takes *actions* in an *environment* (the world it acts in), receives a *reward* (a number saying how good the outcome was), and adjusts its *policy* (its rule for choosing actions) to collect more reward over time. Games and robotics are the classic settings; tuning language models from human preference judgments is a modern one.

Two properties separate this from supervised learning, and both make it harder:

- The feedback is *evaluative*, not *instructive*. A supervisor says “the right answer was cat”. A reward only says “that outcome scored +1” — it never says what the best action would have been.
- The feedback is *delayed*. A chess game’s reward arrives at the end, many moves after the decisive mistake or insight. The agent must work out which of its past actions deserve the credit or the blame.

11.4 What transfers

The map in one table:

Branch	An example is	The feedback is	Typical tasks
supervised	a pair (x, y)	the right answer, per example	prediction, classification
unsupervised	an input x alone	none; loss from inputs only	clustering, compression, anomalies
reinforcement	a state and an action	a reward, possibly delayed	games, robotics, model tuning

What of this chapter survives outside its home cell? Almost all of it. Every branch trains by writing down a loss (its form changes: prediction error, reconstruction error, expected reward with the sign flipped) and walking downhill on it with gradient descent — the loop of §3, the learning rate of §4, the batches of §5. And every branch faces the same enemy: fitting the accidents of its data instead of the pattern — so the vocabulary of §6 through §10 applies everywhere. The taxonomy divides machine learning by *what the data offers*; the engine underneath is shared.

11.5 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
label	another name for the target y ; labeled data comes as pairs
supervised	learning from pairs (x, y) : the right answer is given per example
unsupervised	learning from inputs alone: find structure, no per-example answer
self-supervised	supervised machinery with labels manufactured from the raw data
reinforcement	learning to act from rewards
agent, policy	the learner; its rule for choosing actions
reward	a number scoring an outcome, possibly long after the action

12 Evaluation metrics

The loss is the number training minimizes; it is rarely the number a decision is made on. A cross-entropy of 0.357 tells a person very little about whether a spam filter is good enough to deploy. This closing section builds the standard *metrics* — numbers computed on held-out data (§8) to judge a finished classifier — and shows, on one concrete example, why the most obvious metric fails exactly where it is needed most.

12.1 One thousand emails

A mail service receives 1000 emails: 10 are spam, 990 are not — spam is 1% of the traffic. A situation like this, where one class is much rarer than the other, is called *class imbalance*, and it is the normal situation: fraud among transactions, disease among patients, defects among products are all rare classes.

The obvious metric is *accuracy*: the fraction of decisions that were correct. Now judge the laziest possible filter — it predicts “not spam” for every email, catching nothing:

$$\text{accuracy} = \frac{\text{correct decisions}}{\text{all decisions}} = \frac{990}{1000} = 99\%.$$

Ninety-nine percent accurate, and completely useless. Under class imbalance, accuracy mostly measures the imbalance itself: a model can score it by siding with the common class and ignoring the rare one — which is precisely the class the filter exists to catch. Judging the rare class needs numbers that look at it directly.

12.2 The confusion matrix

Now take a real filter, on the same 1000 emails. It flags 12 emails as spam: 8 of them truly are, 4 are legitimate mail flagged wrongly; and it lets 2 real spam through. Every decision falls into one of four cells, named from the filter’s point of view (spam — the class being hunted — is called the *positive* class, the other the *negative*):

- a *true positive* (TP): flagged spam that really is spam — 8 here;
- a *false positive* (FP): flagged spam that is legitimate mail — 4 here;
- a *false negative* (FN): real spam let through — 2 here;
- a *true negative* (TN): legitimate mail correctly let through — 986 here.

The four counts arranged as a table are called the *confusion matrix*:

	actually spam (10)	actually not spam (990)	row total
predicted spam	TP = 8	FP = 4	12 flagged
predicted not spam	FN = 2	TN = 986	988 passed

The two kinds of mistake have different prices, exactly as §2’s asymmetric-loss discussion anticipated: a false positive buries a real message in the spam folder; a false negative puts spam in the inbox. Which is worse depends on the application — and the metrics below measure the two separately.

12.3 Precision, recall, F1

Each metric is one question about the confusion matrix, computed here for the real filter.

Accuracy — of all decisions, how many were correct?

$$\text{accuracy} = \frac{\text{TP} + \text{TN}}{N} = \frac{8 + 986}{1000} = \frac{994}{1000} = 0.994.$$

Precision — of the emails the filter *flagged*, how many really were spam?

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{8}{8 + 4} = \frac{8}{12} = \frac{2}{3} \approx 0.67.$$

Low precision means many false alarms.

Recall — of the spam that *exists*, how much did the filter catch?

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{8}{8 + 2} = \frac{8}{10} = \frac{4}{5} = 0.8.$$

Low recall means much of the rare class slips through.

F1 — one number combining the two. It is the *harmonic mean* of precision and recall — an average that is high only when *both* are high, so neither can be sacrificed to inflate the other:

$$\text{F1} = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} = \frac{2 \cdot \frac{2}{3} \cdot \frac{4}{5}}{\frac{2}{3} + \frac{4}{5}} = \frac{16/15}{22/15} = \frac{16}{22} = \frac{8}{11} \approx 0.73.$$

Now the comparison that accuracy could not make:

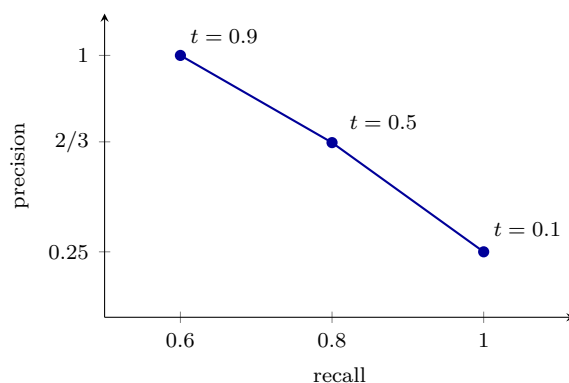
Filter	accuracy	precision	recall	F1
“never flag anything”	0.990	undefined (0/0: nothing flagged)	0	0
the real filter	0.994	2/3	4/5	8/11

By accuracy the two filters differ by 0.004 — almost a tie. Precision, recall, and F1 see what actually happened: one filter does the job and the other does nothing at all.

12.4 The threshold, and the trade-off

A classifier does not output a decision — it outputs a probability (§2’s softmax distribution). The decision is made by a *threshold*: flag the email if $q_{\text{spam}} > t$, with $t = 0.5$ as the default. Moving t moves the filter along a trade-off. Raise it and the filter flags only what it is sure of: fewer false alarms (precision rises), more spam slips through (recall falls). Lower it and the reverse happens. The same filter at three thresholds:

Threshold	flagged	TP	FP	precision	recall
$t = 0.9$	6	6	0	$6/6 = 1$	$6/10 = 0.6$
$t = 0.5$	12	8	4	$8/12 = 2/3$	$8/10 = 0.8$
$t = 0.1$	40	10	30	$10/40 = 0.25$	$10/10 = 1$



the same filter at three thresholds: flagging less buys precision, flagging more buys recall

Sweeping t through *all* values and plotting the whole curve is standard practice. Two curves are common: precision against recall (as above), and the *ROC curve* — the true-positive rate against the false-positive rate, a name inherited from radar engineering. The area under the ROC curve (*AUC*) compresses the whole trade-off into one number: 1 is a perfect ranking of spam above non-spam, 0.5 is coin-flipping.

12.5 Choosing the number

Which metric to report is a decision about costs, not mathematics. If false positives are expensive (a lost customer email), weight precision; if false negatives are (a missed disease), weight recall; F1 when both matter and the classes are imbalanced. This is §2’s lesson returning at the far end of the pipeline: there the choice of loss defined what training pursues; here the choice of metric defines what “good” means when the finished model is judged.

For regression the loss itself usually serves: the MSE, its square root RMSE (back in the target’s own units — an RMSE of 0.5 means predictions typically miss by about half a unit), or the mean absolute error. And whichever metric is chosen, it is computed as §8 prescribed: on the test set, once.

12.6 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
positive class	the class being hunted (spam here); the other is negative
TP, FP, FN, TN	the four cells of the confusion matrix
class imbalance	one class much rarer than the other
precision	$TP/(TP + FP)$: of the flagged, how many were real
recall	$TP/(TP + FN)$: of the real, how many were caught
F1	the harmonic mean of precision and recall
threshold t	the probability above which the classifier flags
ROC, AUC	the rate-vs-rate curve over all thresholds; the area under it